

# Embedded Operating System

LIAO Yong (Alan)

## Embedded OS 2024

LIAO Yong (Alan)

UESTC Real-Time System Lab

### *Table of Contents*

- Introduction
- Kernel of ERTOS
- Hardware System
- Software System
- 附录
  - BootLoader.s
  - 24年半期试题
  - 24年期末试题回忆版

## - 1 Introduction -

# Embedded Operating System

LIAO Yong (Alan)

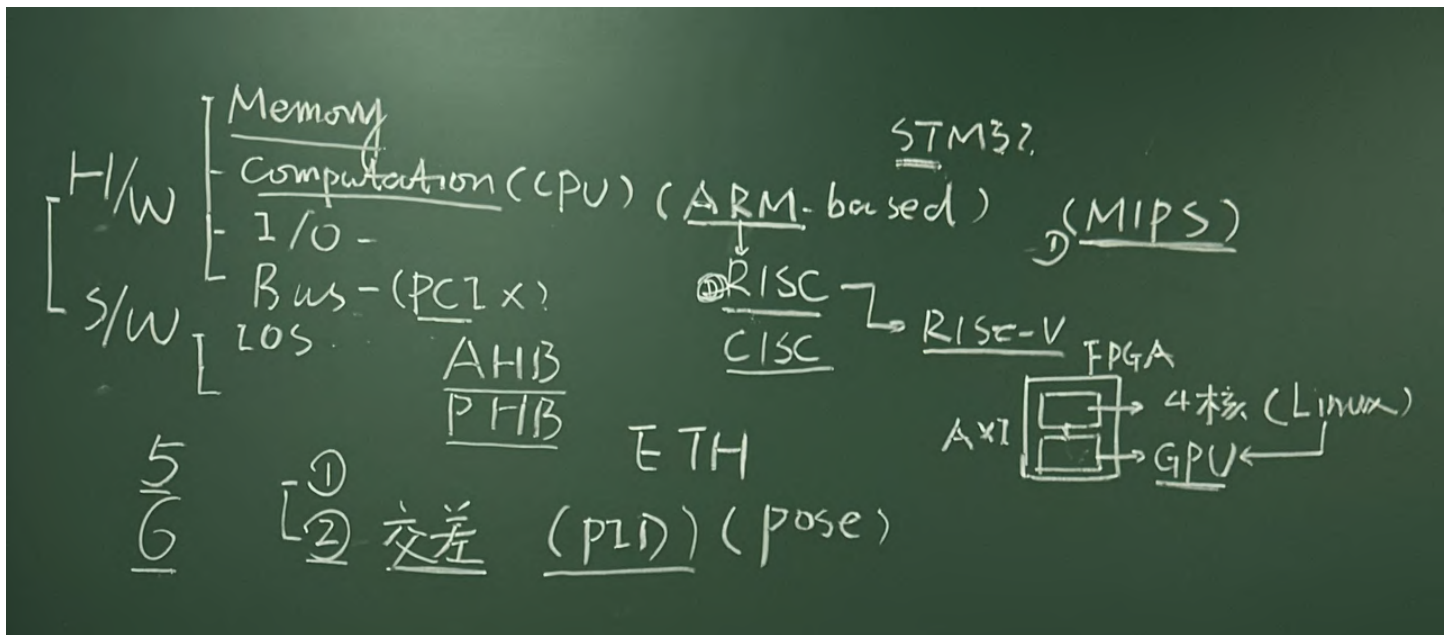
Real-Time System Lab

LIAO Yong

 1 Introduction.ppt

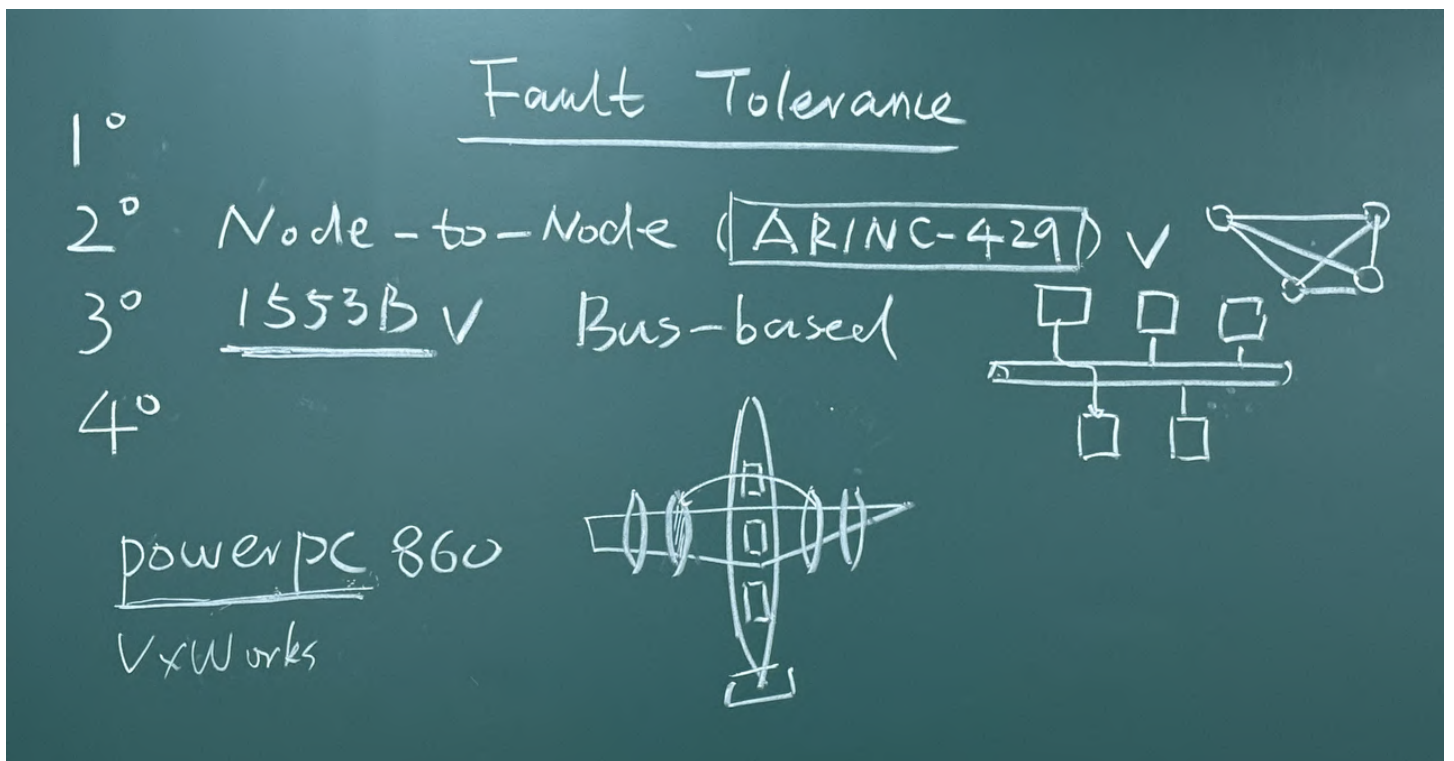
## P4

- H/W
  - 冯诺伊曼架构
    - Mem
    - CPU（手机：ARM-base/MIPS） R： RISC-V
    - I/O
    - Bus
- S/W
  - IOS、 Android...



## P8

热备份



## P10

Vxworks——硬实时操作系统

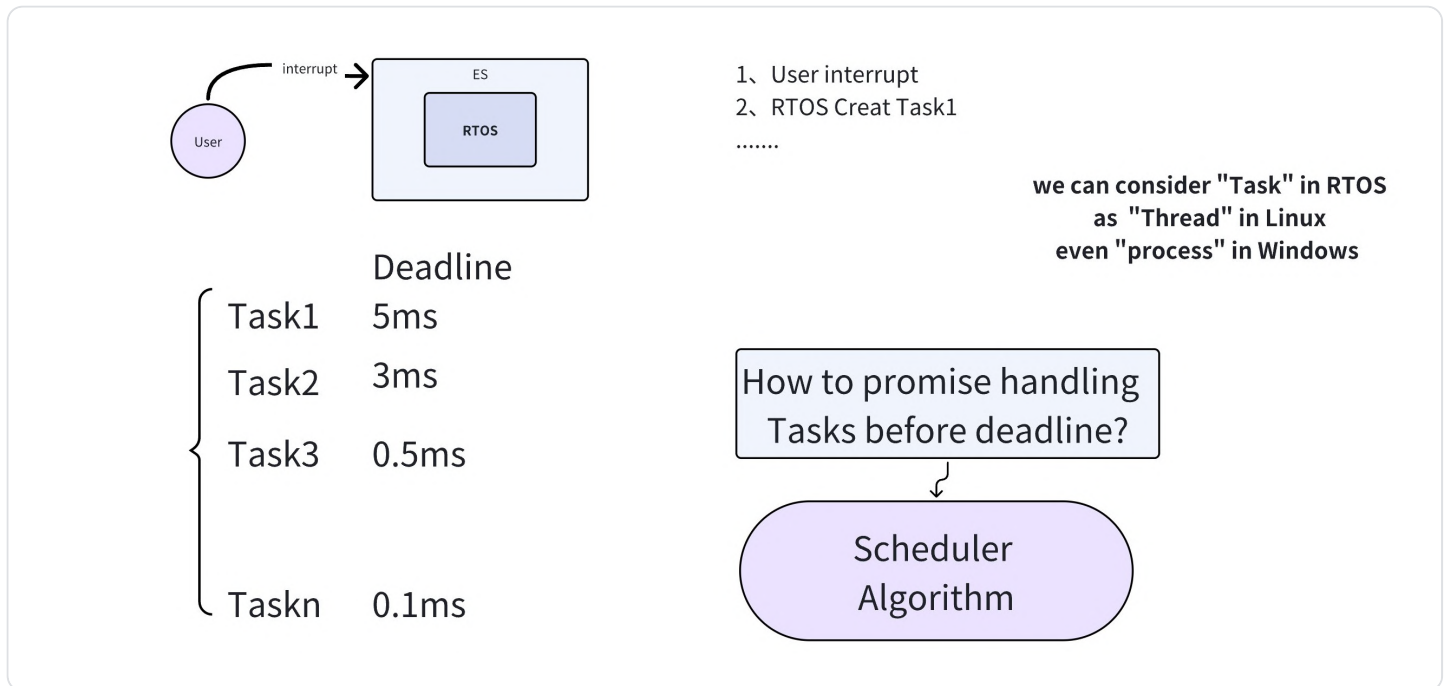
- Built in Test (优先级非常高) & 守护进程
- DeadLock 调度算法问题导致优先级改变, 出现死锁导致系统重启

## P11

IEEE Pervasive Computing杂志

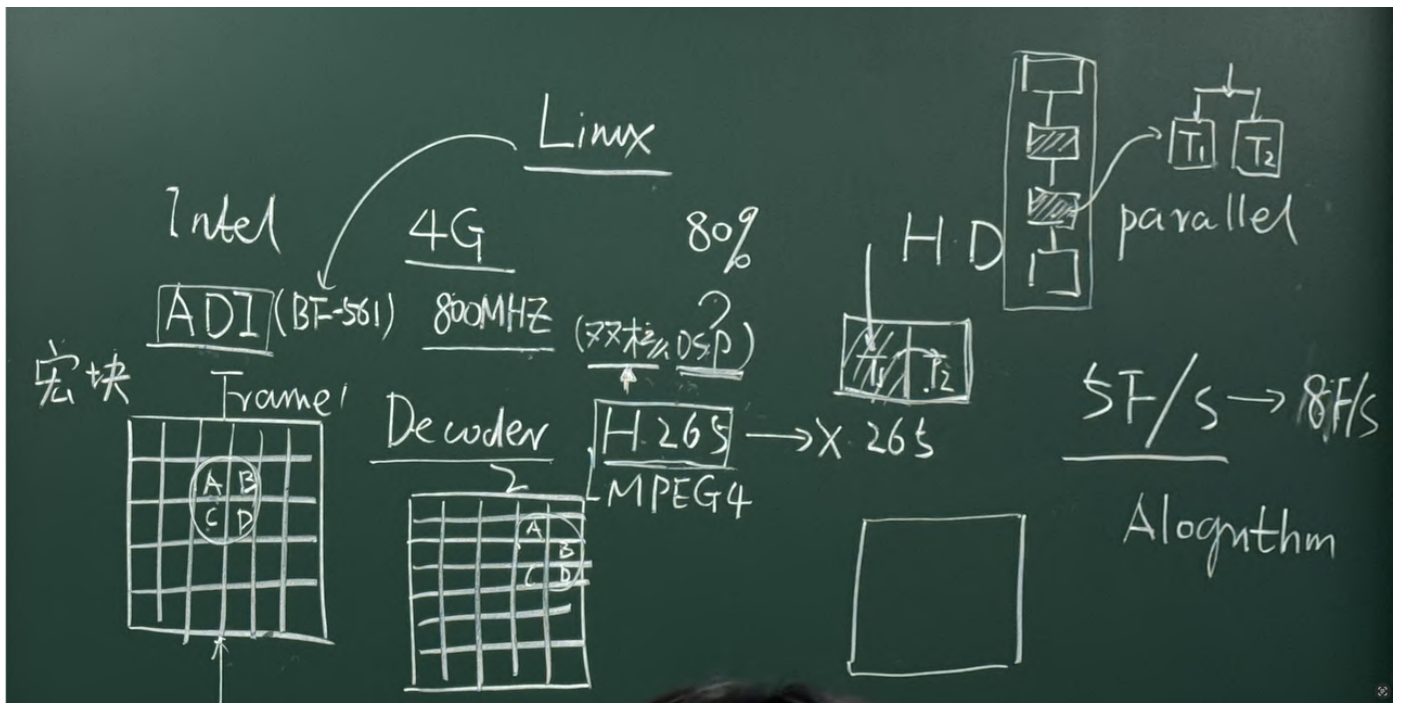
# P13

- What is RTOS?
  - Timely response (Timely response)



- **Hard RT**
  - **保证在Deadline前完成任务。** Realiability!!! 否则造成灾难性后果
  - 导弹拦截系统
- **Soft RT**
  - 有实时性要求，但并不至于到苛刻的标准，它允许在尽可能低的Deadline miss Ratio情况下实现整个系统的流畅运行
  - Player —— 要求：至少帧率>25
    - 解析一张画面的实时性要求：40ms内
    - 本地性能可能不够，网络也不一定稳定



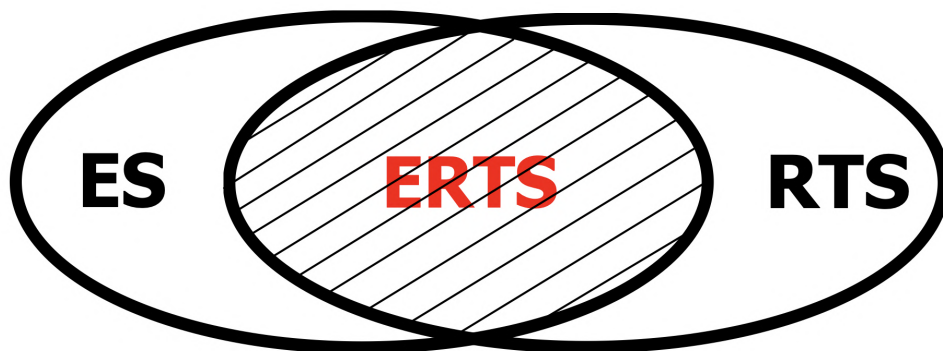


编码算法：计算每一帧之间的变化即可——大幅缩小视屏体积，提升解码性能

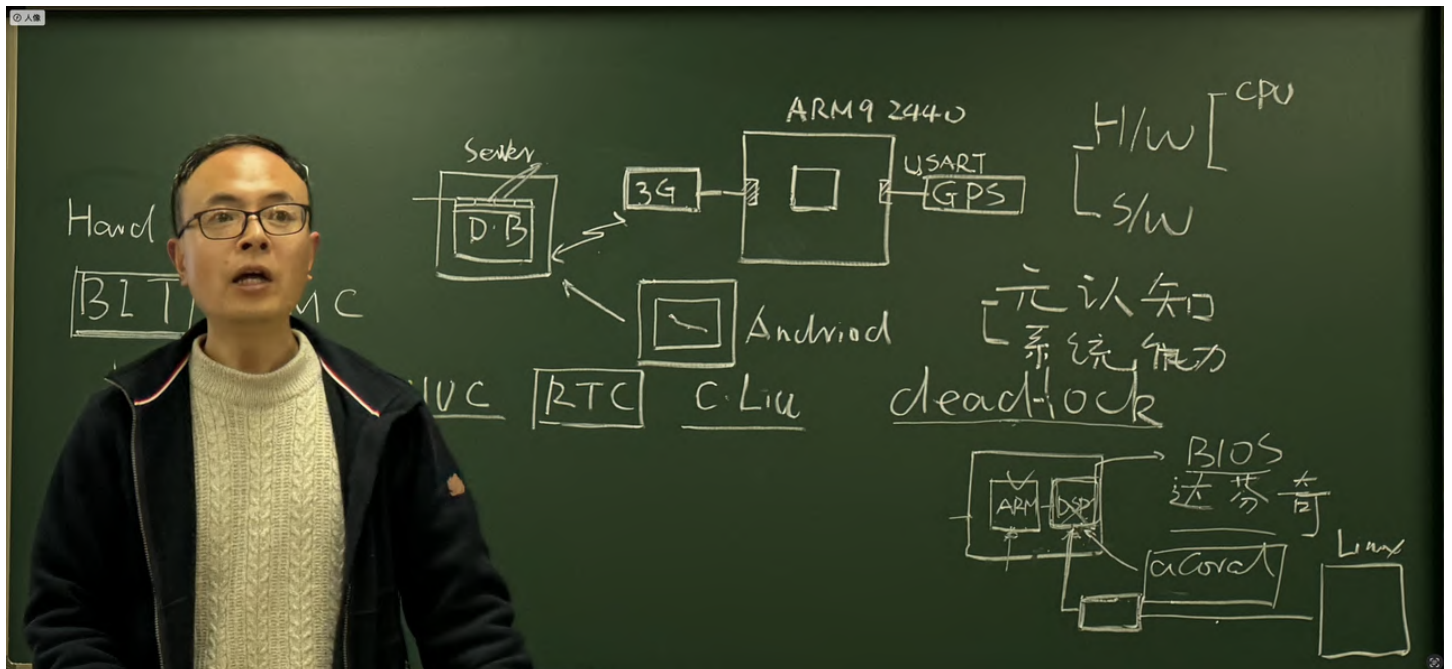
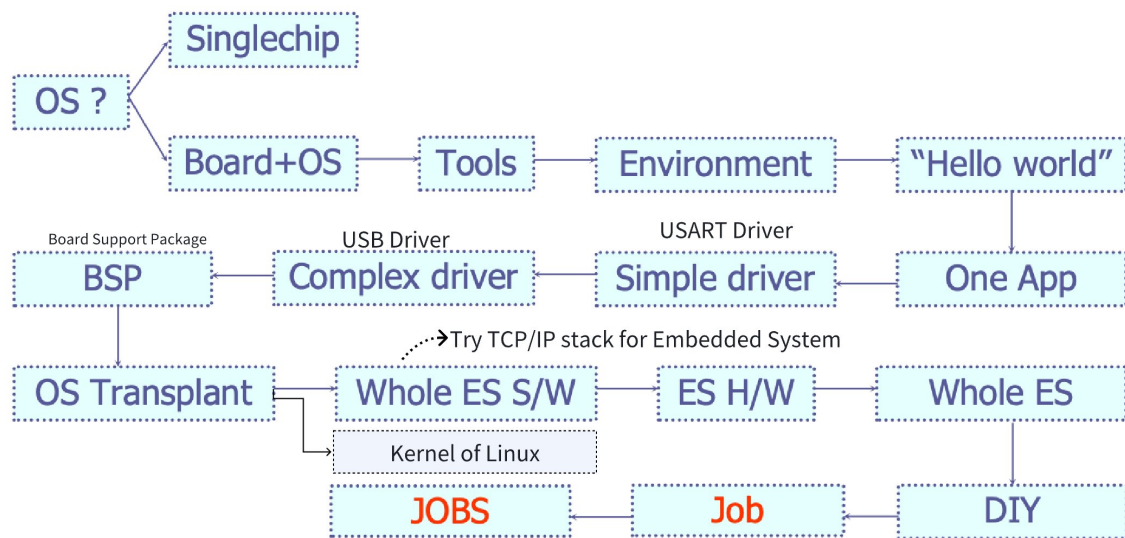
- 运动估计——可以去下载H.264的代码，分析它的代码流程图——如何把串行的优化为并行

P14

ES  $\neq$  RTS



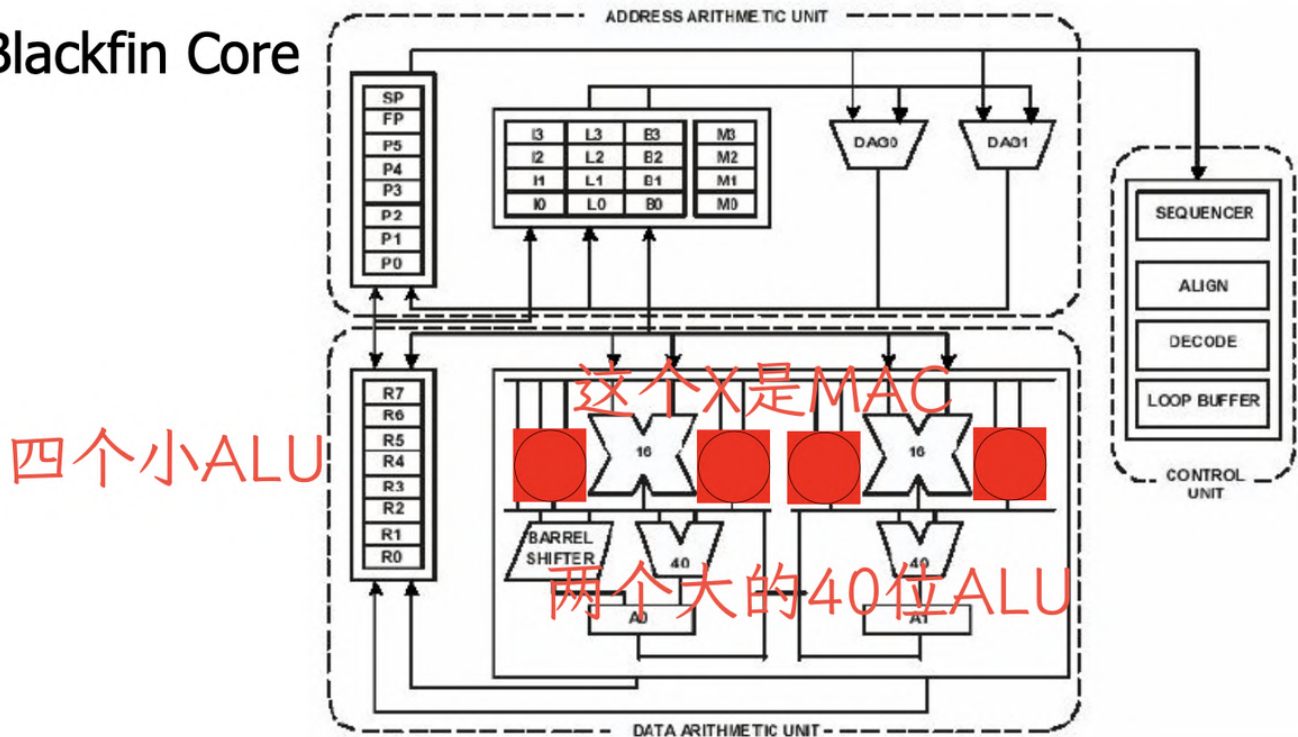
**ES**: Embedded Systems  
**RTS**: Real-Time Systems



P35

四个ALU V型的计算

## ■ Blackfin Core



## P15

- Optimize 如何优化性能
  - Implementation
    - Algorithm Parallel
    - Instruction set

## ■ Application specific

- **Specialize** the design for specific application 😊
- **Optimize** the design for specific application 😊

## ■ Have to worry about both hardware and software

## ■ Have to worry about non-functional constraints

- Real-time 😊
- Memory
- Power
- Cost
- Reliability (**Security, Safety.....**)

Just functionally working is NOT enough!

Security: 信息安全; Safety: 防危性 (电梯安全运行)

## P37

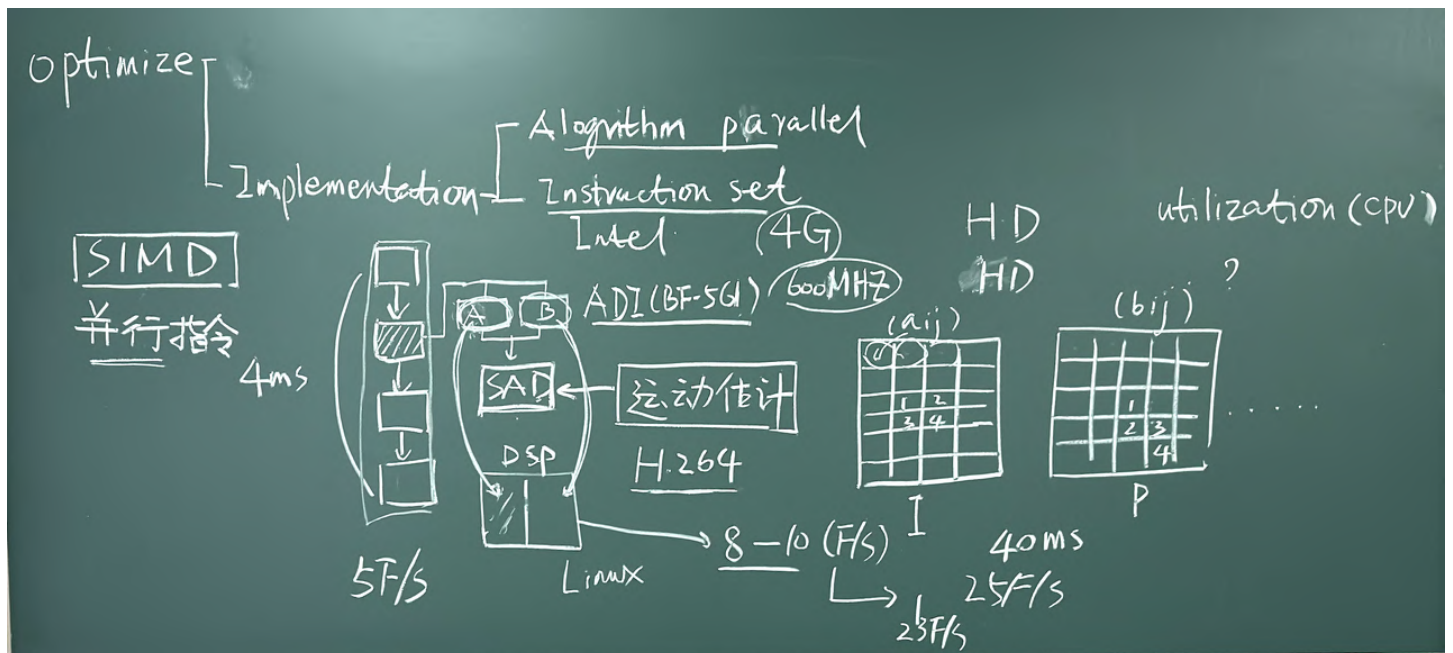
$$(r_0, r_1) = \text{ByteOp16P}(r_0, r_1)$$

BF561的一个指令，单指令多数据流

|       | 31—24 | 23—16 | 15—8  | 7—0   |
|-------|-------|-------|-------|-------|
| $r_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
| $r_1$ | $Z_3$ | $Z_2$ | $Z_1$ | $Z_0$ |

在一个指令周期里把  $r_0$   $r_1$  两个32位的寄存器做4次

|       |             |    |             |
|-------|-------------|----|-------------|
| $r_0$ | $Y_1 + Z_1$ | 运算 | $Y_0 + Z_0$ |
| $r_1$ | $Y_3 + Z_3$ |    | $Y_2 + Z_2$ |



## P38

这种有两个下划线的，是并行指令。

SAA——相减、求绝对值、累加



# 并行指令，取指与取数据同时进行！

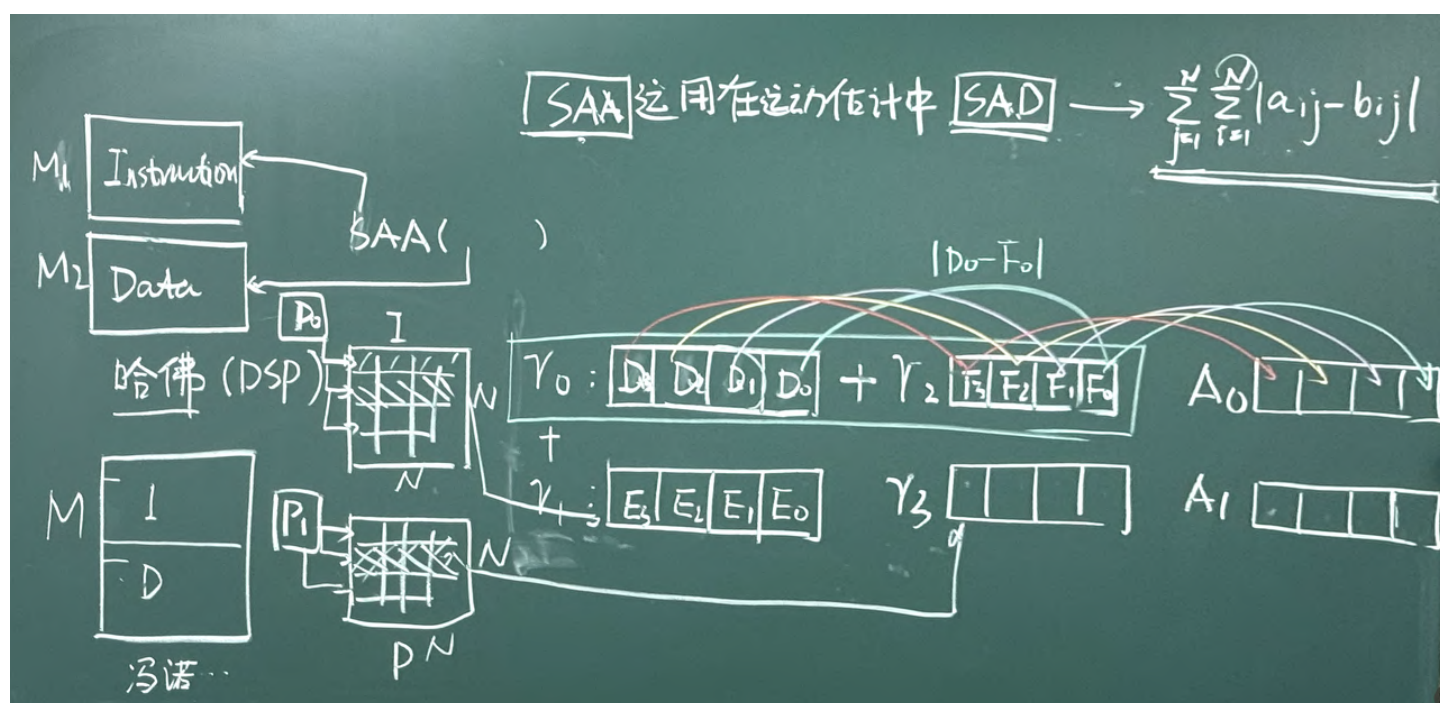


$SAA(r_{1:0}, r_{3:2}) \parallel r_1=[P_0++] \parallel r_3=[P_2++]$

是哈佛架构特有的优势——Instruction与Data分存储器存放

|       | 31—24      | 23—16      | 15—8       | 7—0      |
|-------|------------|------------|------------|----------|
| $r_0$ | $a(i,j+3)$ | $a(i,j+2)$ | $a(i,j+1)$ | $a(i,j)$ |
| $r_2$ | $b(i,j+3)$ | $b(i,j+2)$ | $b(i,j+1)$ | $b(i,j)$ |

|       | 39—20                   | 19—0                    |
|-------|-------------------------|-------------------------|
| $A_1$ | $ a(i,j+3) - b(i,j+3) $ | $ a(i,j+1) - b(i,j+1) $ |
| $A_0$ | $ a(i,j+2) - b(i,j+2) $ | $ a(i,j) - b(i,j) $     |



## P16

- 轮询系统 Pulling Loop
  - 可是完全不能和用户交互。唯一的方式是修改代码
- 前后台系统
  - 一种与用户交互的方式：触发中断
    - 可以是按钮等
- 多任务系统 (OS)

? ucOSii有区分用户态与核心态吗

- 沒有。這是一個大內核結構的操作系統

## - 2 Kernel of ERTOS -

1

University of Electronic Science & Technology of China (UESTC)

# Embedded Operating System

LIAO Yong

Real-Time Systems Lab

LIAO Yong

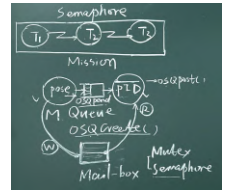
P 2 Kernel of ERTOS.ppt

## P3 INTRO

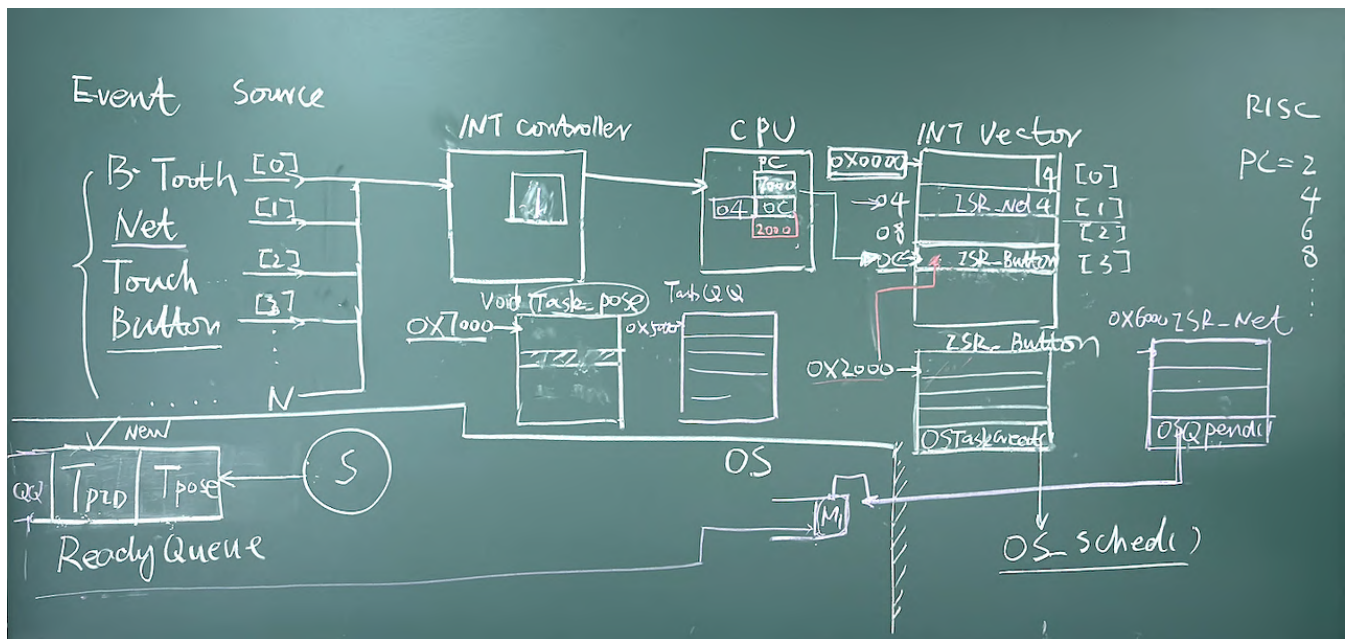
### Overview

- Task management [**Scheduling Policy**]
- Synchronization (同步)、Communication (通信)、Mutex (互斥)  
[**Coordinate mechanism**]

- 不同任务之间可以独立、也可以有先后的顺序关系
  - 信号量、消息队列、共享内存（互斥访问很重要）、邮箱



- Memory Management [Inventory management mechanism]
- Interruption & Time management [Event dealing mechanisms(Event-trigger&Time-trigger)]
  - Event-trigger—**Interruption**
    - BT、Net、Touch、Buttom

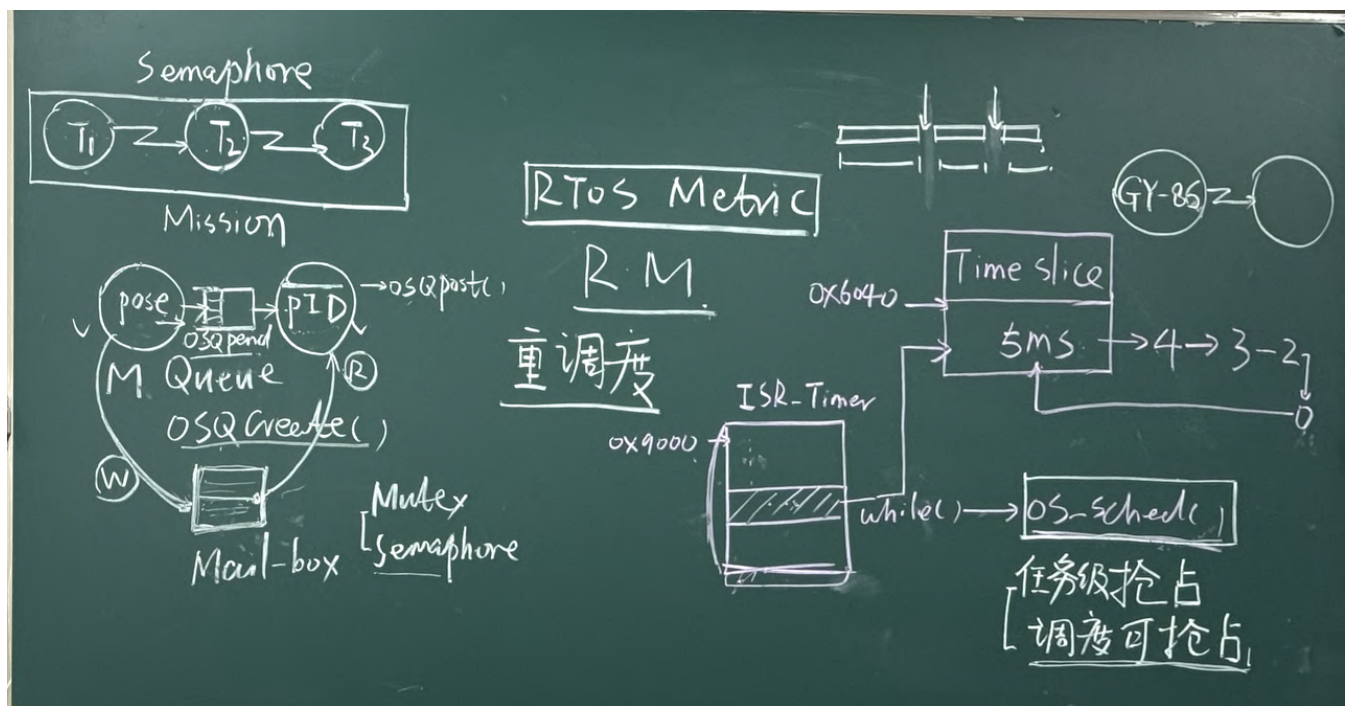


事件触发机制简图。简述了事件触发中断，引发OS重调度的过程

### ? OS在调度的时候可以被中断打断吗?

- 任务级抢占&调度级抢占
- 这个就体现了RTOS的实时性能。假如操作系统在调度的时候来了极其紧迫的需求，是否能够及时响应?
- OS在调度时，会有开关中断的操作





## P4 KERNEL

### Q1 What does the kernel do?

- Scheduling

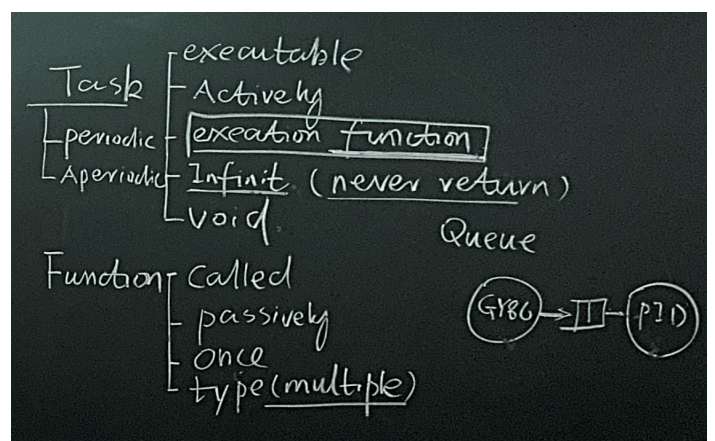
### Q2 What does the kernel scheduling?

- Task

### Q3 Task 的特征，与 Function 的区别

#### • Task VS Fuction

- 任务：可被调度 —— 函数：可被调用
- 任务：主动运行 —— 函数：被动运行
  - 任务不主动删除/挂起自己，要是进了就绪队列，就总会被操作系统调度出来。
  - 因此，**任务是函数的一次执行**，不主动删除自己就会主动的进入就绪队列等待被调度，将**Infinite**的执行下去。
- 任务的返回类型是Void
- 任务 拥有函数、栈、计算机资源等等



任务与函数的对比图



- Task (Thread) 是操作系统调度的基本单位
  - Task的特点
    - 并发性
    - 异步独立&同步依赖
    - 动态性
  - Concurrency (Simultaneousness)
  - Independence & Asynchrony (dependence & Synchrony)
  - Dynamic
- 

## P5 TASK

### Q1 How to describe a task?

➤ How to describe a task 😊😊😊

- Task:
  - 可执行函数
  - 栈
  - TCB

😊 P133->P4

- P133
  - A task is a **executable software entity** specialized to perform a special action (Enemy Detect, Danger Evaluation, Missile Launching.....), which includes a **sequential code**, and **relative data** and **computer resource**.
  - A task is an **infinite loop function**. it looks just like any other C function containing a return type and an argument, but it must **never return**. The return type of a task must always be declared to be **void**.

P133 任务的两种定义

- 然后跳转到 P4,

😊 P89~P94

操作系统任务调度简要逻辑

## • 一个任务时

```
void YourTask (void *pdata)
{
    for (;;)
    {
        /* USER CODE */
        .....
        OSMsgboxPend();
        OSQPend();
        OSSemPend();
        .....
        OSTaskSuspend(OS_PRIO_SELF);
        OSTimeDly();
        OSTimeDlyHMSM();
        /* USER CODE */
    }
}
```

P89 一个任务的函数部分的简要构成

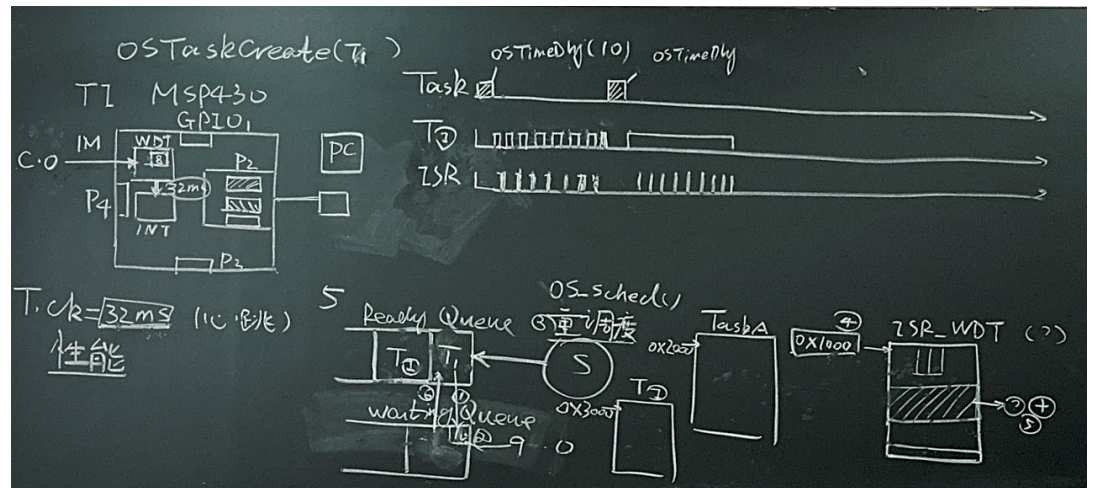
```
void Task1(void *pdata)
{
    WDTCTL = WDT_MDLY_32; /*设置时钟节拍间隔为32ms*/
    IE1 |= BIT0; /* 开看门狗定时器中断*/
    P2SEL &= BIT1; //引脚功能选择器
    P2DIR |= BIT1; //方向寄存器, 1为输出模式

    while(1)
    {
        P2OUT |= BIT1; //LED亮
        OSTimeDly(10); //延时10个时钟节拍
    }
}
```

P90 一个简单的点灯任务（基于 MSP430）

## • Tick=32ms是怎么来的？

- 乙方思维：怎么从技术上实现
- 甲方思维：需求，性能与成本的综合考量



P90 代码作为 Task1，附上 IDLE 后操作系统的调度逻辑

## • OSTimeDly(10)

- 这个任务到哪儿去了

## ? 这里把延时设计为OSTimeDly()与之前裸机程序使用的延时函数有什么区别？

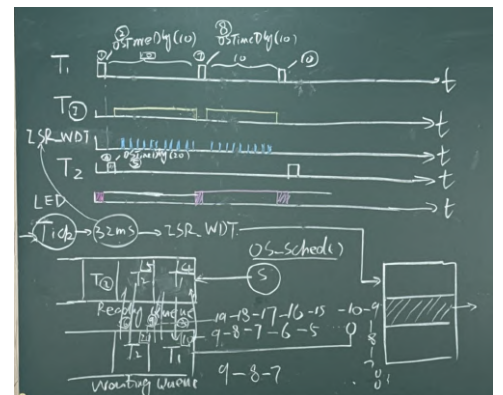
- OSTimeDly()在执行时，CPU资源从一个任务转交给其它任务执行
- 裸机的延时函数在执行时，CPU资源全部分配给延时函数，无法执行其它函数。

## • 来了第二个Task：关灯

```
void Task2(void *pdata)
{
    WDTCTL = WDT_MDLY_32; /*设置时钟节拍间隔为32ms*/
    IE1 |= BIT0; /*开看门狗定时器中断*/
    P2SEL &= BIT1; //引脚功能选择器
    P2DIR |= BIT1; //方向寄存器, 1为输出模式

    while(1)
    {
        P2OUT &= ~BIT1; //LED灭
        OSTimeDly(20); //延时20个时钟节拍
    }
}
```

P91 一个简单的灭灯任务



引入两个任务后，操作系统的调度逻辑

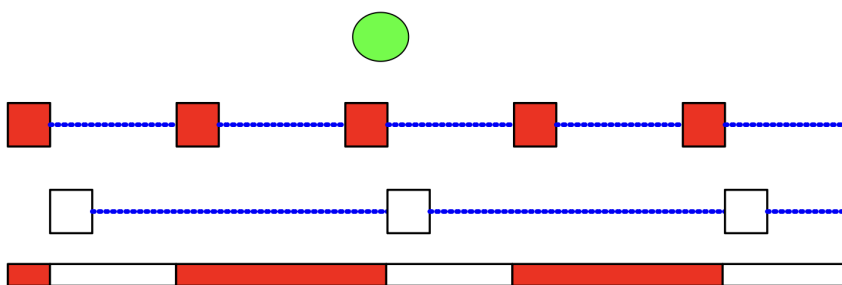
? 如果一个Task的执行时间大于一个Tick怎么办?

- 运行时，该任务依然存在于就绪队列
- 每一个Tick到来时，都会来执行ISR\_WDT 中断服务程序

## 📌 关键点

- IDLE 空闲任务
- 一般任务、IDLE 任务与中断服务程序的关系
- 操作系统中的就绪队列、等待队列

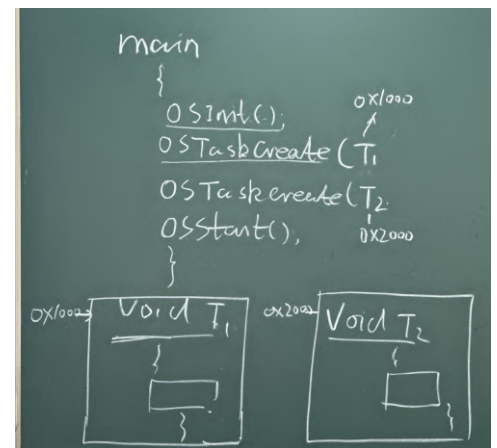
## 📌 Difference between Process&Thread(Task)



Where and when to create the two tasks?

How to start the two tasks ?

P92



University of Electronic Science & Technology of China (UESTC)

What's the difference between task and process?

```
int i=1;

test(){
    sleep(10S); /*waiting for the execution of "i++"*/
    printf("%d",i);
}

int main(){
    create_process(test,.....);
    i++;
}
```

在main 中创建子进程

```
int i=1;

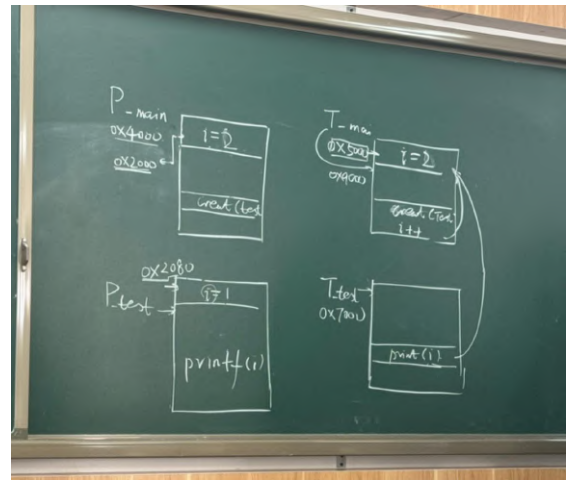
test(){
    sleep(10S); /* waiting for the execution of "i++"*/
    printf("%d",i);
}

int main(){
    create_task(test,.....);
    i++;
}
```

在 main 中创建子任务

- 进程是资源管理的基本单位，线程是操作系统调度的基本单位
- 进程之间内存空间彼此隔离独立，无法互相访问，线程之间内存空间可互相访问
  - 进程之间的内存会由MPU/MMU硬件单元管理，使得他们之间互相隔离
  - 一个进程崩溃了，其他进程不受影响；一个线程崩溃，所属同一个进程的其他线程将受到影响





进程与线程可访问内存资源示意图

## 😊 P134——TCB

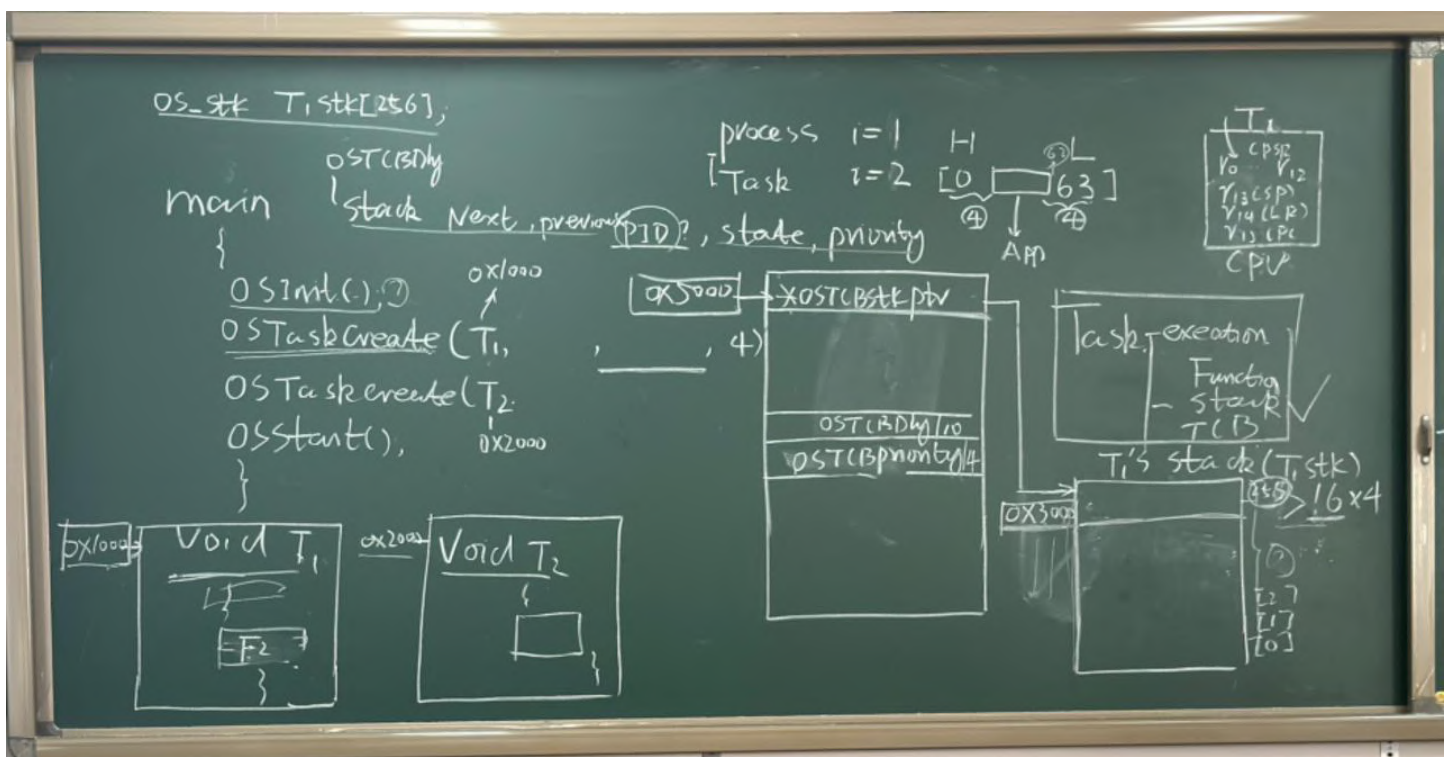
创建任务的本质就是在任务的可执行函数、TCB 与栈之间建立联系。创建好后，它才能进入就绪队列。

### • 一个任务创建的栈

#### ◦ 最小的大小？

- CPU寄存器有16个，所以至少是16\*4个bit的大小（任务切换时，要保存现场，把这个任务执行时CPU里的东西给存下来）
- 函数内局部变量
- 函数间调用关系需要的动态链接、静态链接

#### ◦ 栈的地址？





## TCB 的构成

### 1. `*OSTCBStkPtr`

- 任务所拥有的可执行函数在执行时需要一个栈空间。这个栈并不是芯片启动时在 startup.s 中定义的栈（那是给 OS 自己用的）。这个栈的大小由程序员评估确定（芯片寄存器、函数局部变量、函数调用关系）
  - 类似于 `int a[128]` 一样，在 `μc/OS-ii` 中也是通过一个 typedef 出来的 `OS_STK` 初始化一个数组来实现：`OS_STK Task1_stk[128]`

### 2. `*OSTCBNext` & `*OSTCBPrev`

- 指向前后 TCB 的指针，这样一来就可以以双向链表实现就绪队列了

### 3. `OSTCBState`

- 描述任务状态。

### 4. `OSTCBDelay`

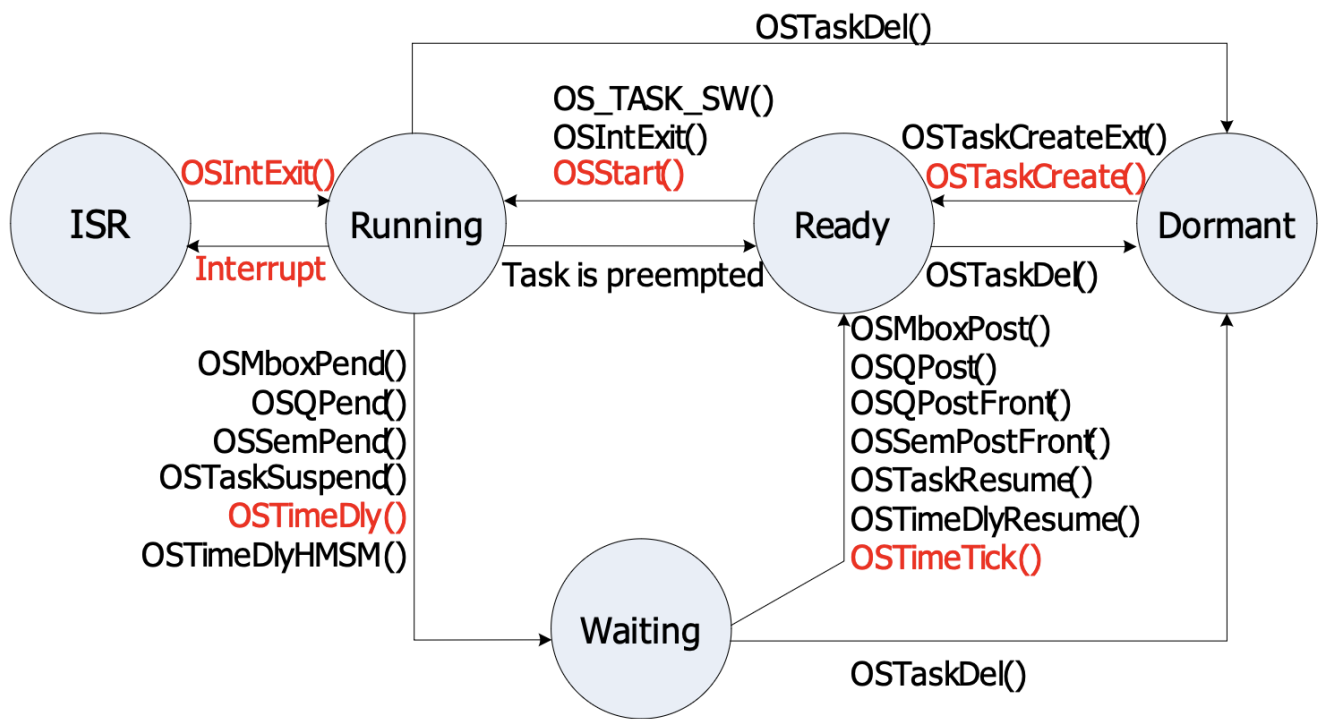
- 这个就是 Task1 点灯延迟 10 秒时，操作系统对这个任务进行计时的地方。它从一个初始值不断递减，到一定值后该任务将从等待队列进入就绪队列。
- 这个值是由位于 ISR\_WDT 中断服务程序中的 `OSTimeTick()` 来进行管理。

### 5. `OSTCBPrio`

- 描述任务优先级。`μc/OS-ii` 中，优先级可配置为 64 种或者 128 种。对于 64 级优先级而言：0~3 与 60~63 为操作系统所用。
- 任务间优先级不得相同。因此，可以仅仅依靠优先级来确认任务，TCB 中不用再设置任务 ID

### 6. `OSTCBX`、`OSTCBY`、`OSTCBBitX`、`OSTCBBitY`

- 为了支撑实时性，需要一种  $O(1)$  的任务调度算法，使得查找任耗时与任务数量无关。这种算法用到了一个数据结构：**Priority Bit Map**

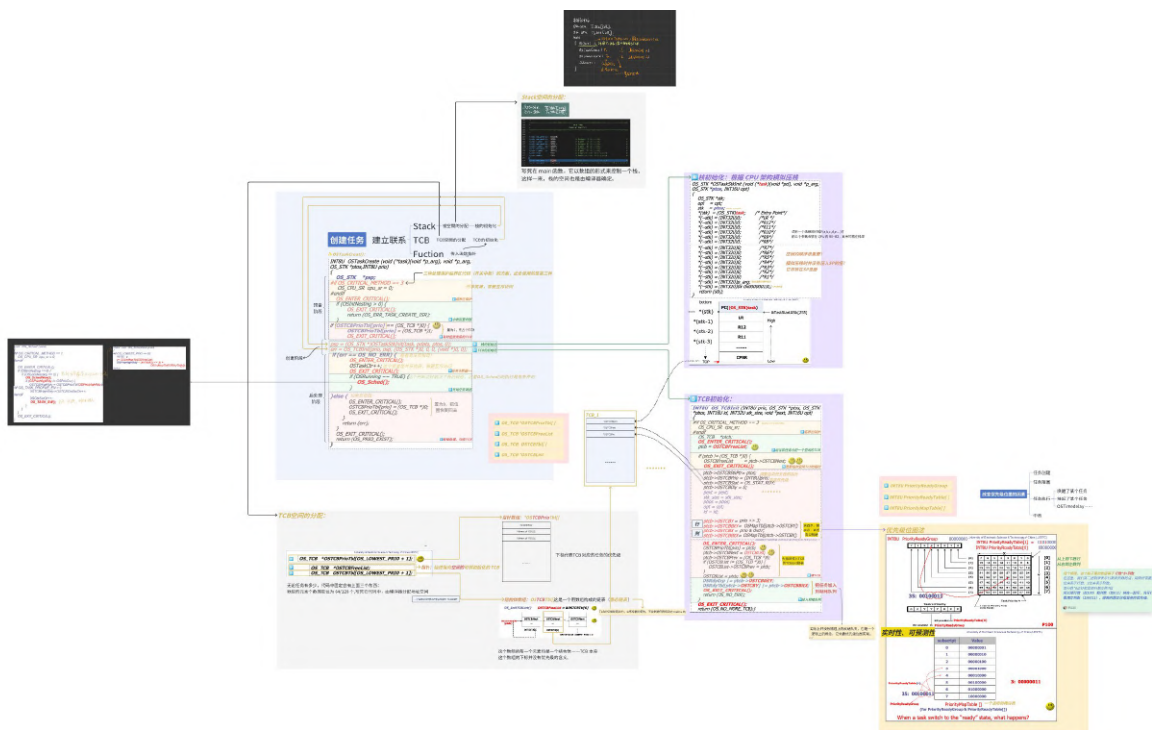


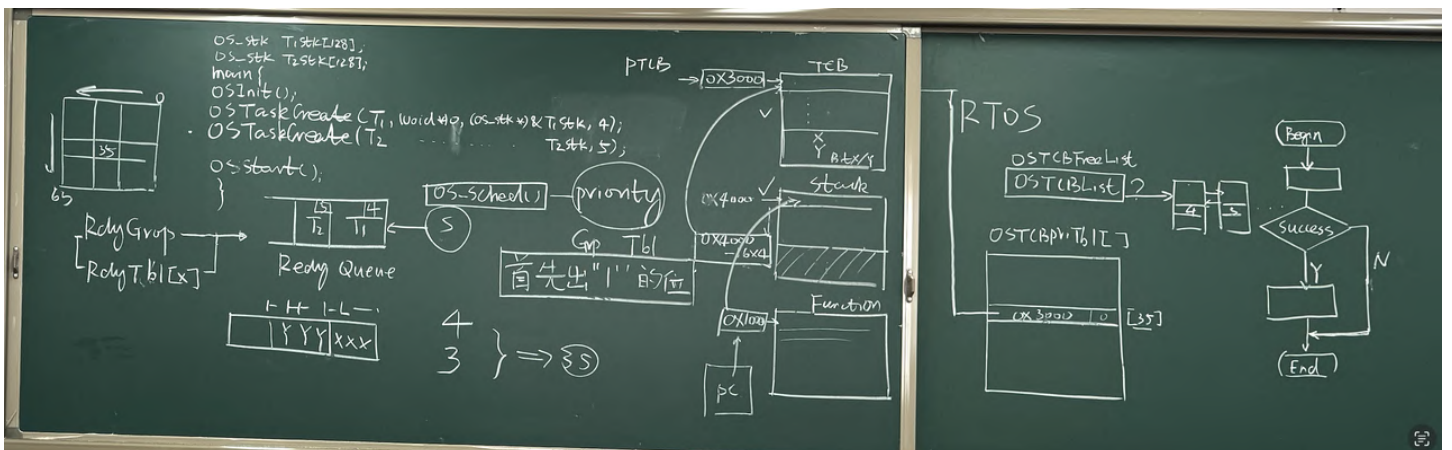
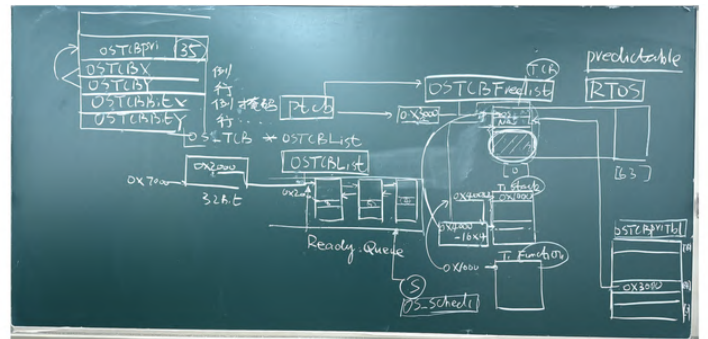
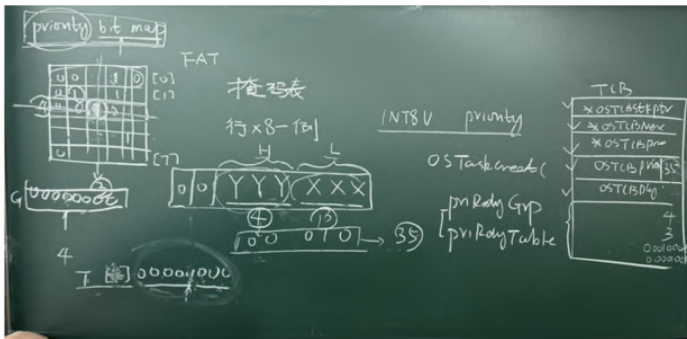
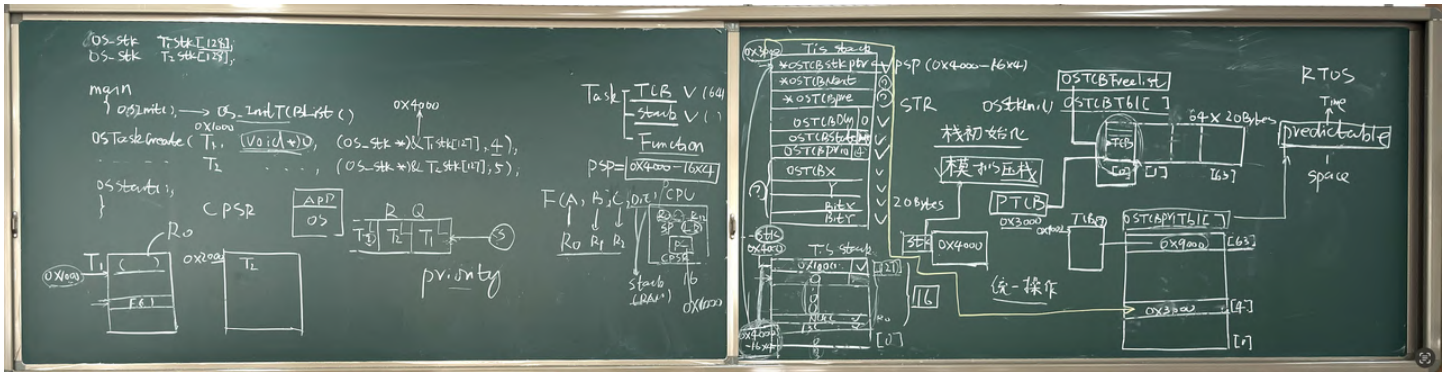
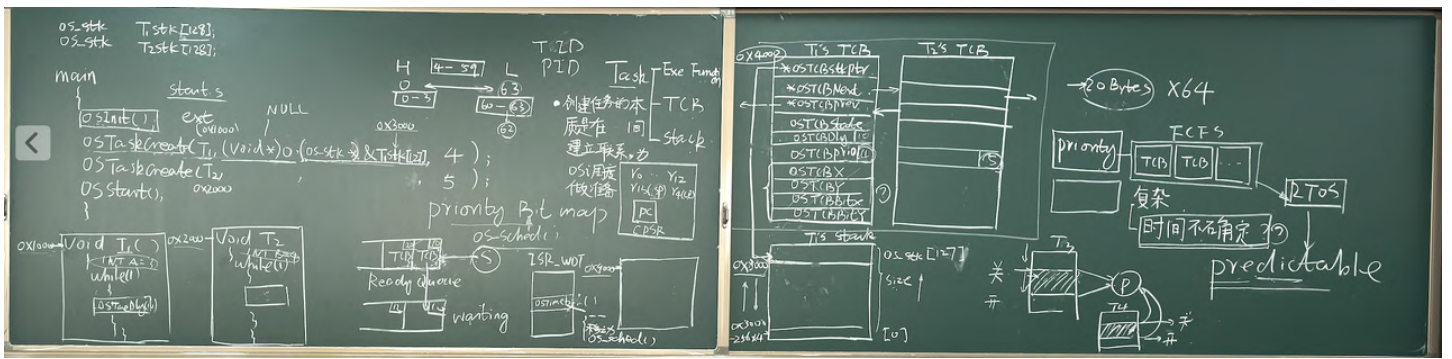
P137 RTOS 将不停地维护这个状态机

Q2 How to create a task in  $\mu\text{C}/\text{OS II}$ : ➤ How to create a task in uc/O! 😊 ?

## 😊 创建任务: OSTaskCreat()

- 那创建一个任务需要什么参数呢? 函数怎么调用?
  - OSTaskCreate 的四个参数: 函数指针、指向需要传递的参数的指针、任务栈顶指针、任务优先级





? 任务创建的时候，栈的初始化是不是可有可无的? ❌

- 在任务创建后就会进入就绪队列
- 可是就绪队列里不只是有刚创建好的任务，还有其他已经执行过的任务



- 他们的区别就是栈中内容
- 可操作系统把就绪队列中任务提出进入执行态，需要执行一系列操作，其中就要把栈中的一部分数据压入到 CPU 中（由汇编代码实现）
- 压入到CPU中的操作是统一的。如果新建的任务不对栈进行一定初始化，PC 等关键值就会出错，程序会跑飞。

## ? 指针数组：OSTCBPrioTbl[] 的作用和意义是什么 ⚠

- OSTCBPrioTbl[] 是一个指针数组。对于 OSTCBPrioTbl[63] 而言，其值的意义是：一个指向优先级为 63 的任务的 TCB 的指针。
- 设计这个数组的作用是为了在的时间复杂度内找出某优先级任务对应的 TCB，是一种牺牲空间、换得时间的，在优先级与任务建立联系的方式，OS\_Schdu() 函数的返回值是任务的优先级。这时就可以通过数组的取下标操作迅速找到这个任务对应的 TCB，随后立即定位到该任务的栈，随即开始恢复该任务的运行环境

## 😊 任务调度：OS\_Sched()

The image contains several components related to task scheduling:

- Code Snippets:**
  - `void OS_Sched(void)`: The main scheduling function, showing critical section entry, priority search, and context switch.
  - `static void OS_SchedNew(void)`: A helper function for finding the highest priority task in the ready queue.
  - `OSTCBPrioTbl`: A pointer array used to map priorities to task TCBs.
- Diagram:** A diagram titled "优先级决策表" (Priority Decision Table) showing a grid of task priorities and their corresponding TCB pointers. It illustrates how the scheduler finds the highest priority task.
- Annotations:** Various notes in Chinese explaining the logic, such as "不允许处于嵌套中!" (Not allowed to be nested!), "性能统计" (Performance statistics), and "用空间换时间" (Using space to save time).

The image shows hand-drawn diagrams and code snippets illustrating task scheduling:

- Code Snippets:**
  - `OS_TaskCreate`: Function to create a task, including setting up the task structure and adding it to the ready queue.
  - `OS_Start`: Function to start a task, including setting up the task's initial state and adding it to the ready queue.
- Diagram:** A diagram showing the task scheduling process. It includes a "Ready Queue" (就绪队列) with tasks represented as boxes. A "Priority" (优先级) is assigned to each task, and the task with the highest priority is selected for execution. The diagram also shows the task's state transitions (Ready, Running, etc.) and the role of the "PC" (Program Counter) in task execution.

3.20

Q3 How to Schedule tasks in Ready Queue?

➤ How to schedule tasks in the ready queue 😊?



- 156



- 
- The diagram illustrates the context switch process in an RTOS, showing the flow from a task function to a context switch routine, which then saves the current task's state and restores the next task's state.
- Context Switch Process:**
- Save Current Task State:** The current task's state (PC, registers, etc.) is saved into its TCB.
 

```

      OSCbSw:
      STMFD SPI, {LR}          ;PC 压入任务堆栈, 然后得到PL1为任务调度策略PCP
      STMFD SPI, {R0-R12, LR}  ;R0-R12 LR
      MRS  R0, CPSR            ;Push CPSR
      STMFD SPI, {R0}
      ;
      OSTCBrCur->OSTCBSPtr = SP
      ;
      LDR  R0, =OSTCBrCur
      LDR  R0, {R0}
      STR  R0, [R0, #4]
      BL  OSTaskSwHook
      OS_EXT OS_TCB *OSTCBrCur;
      
```
  - Update Task Priority:** The task's priority is updated in the TCB.
 

```

      LDR  R0, =OSTCBrCur
      LDR  R0, {R0}
      STR  R0, [R0, #4]
      BL  OSTaskSwHook
      OS_EXT OS_TCB *OSTCBrCur;
      
```
  - Update Current Task:** The current task's state is updated in the TCB.
 

```

      LDR  R0, =OSTCBrCur
      LDR  R0, {R0}
      STR  R0, [R0, #4]
      BL  OSTaskSwHook
      OS_EXT OS_TCB *OSTCBrCur;
      
```
  - Update Task Priority:** The task's priority is updated in the TCB.
 

```

      LDR  R0, =OSTCBrCur
      LDR  R0, {R0}
      STR  R0, [R0, #4]
      BL  OSTaskSwHook
      OS_EXT OS_TCB *OSTCBrCur;
      
```
  - Restore Next Task State:** The next task's state is restored from its TCB.
 

```

      LDR  R0, =OSTCBrHighRdy
      LDR  R0, {R0}
      STR  R0, [R0, #4]
      BL  OSTaskSwHook
      OS_EXT OS_TCB *OSTCBrHighRdy;
      
```
  - Restore Next Task State:** The next task's state is restored from its TCB.
 

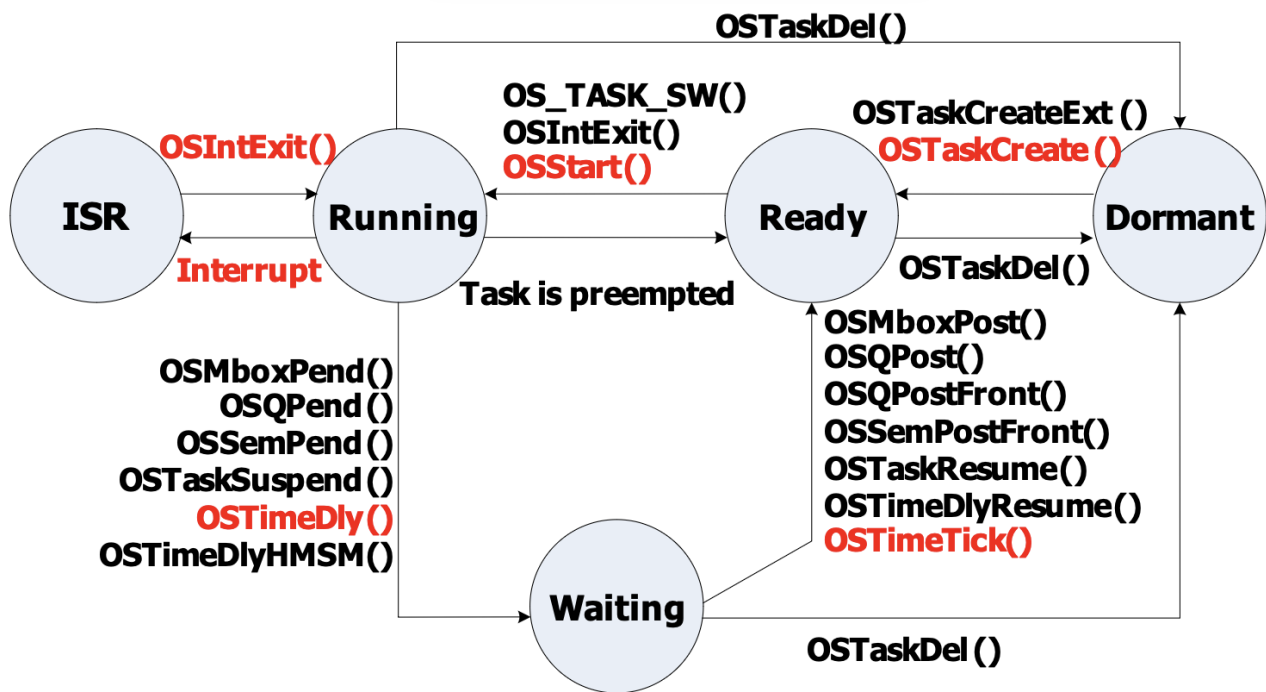
```

      LDR  R0, =OSTCBrHighRdy
      LDR  R0, {R0}
      STR  R0, [R0, #4]
      BL  OSTaskSwHook
      OS_EXT OS_TCB *OSTCBrHighRdy;
      
```
- The diagram also shows the state of the system with multiple tasks and the current task being switched out. The process is numbered 1 through 6, showing the sequence of operations from saving the current task's state to restoring the next task's state.

➤ Which events will trigger the context switch? 😊

**Scheduling Point**

- OS\_Sched()函数分布于操作系统内核各处，下图中的横线上的函数、事件中便会调用OS\_Sched()，触发任务切换：



编写操作系统的过程就是构建这么一个有限状态自动机

- 详见教材3.1

## Q5 Critical section accessing during sched

➤ Critical section accessing during sched 😊😊

- 临界区处理

```
#if OS_CRITICAL_METHOD==1
```

```
#define OS_ENTER_CRITICAL() (Cli())
```

```
#define OS_EXIT_CRITICAL() (Sti())
```

```
#elseif OS_CRITICAL_METHOD == 2
```

```
#define OS_ENTER_CRITICAL() (PushAndCli())
```

```
#define OS_EXIT_CRITICAL() (Pop())
```

```
#elseif OS_CRITICAL_METHOD == 3
```

```
#define OS_ENTER_CRITICAL() (cpu_sr = OSCPU_SaveSR())
```

```
#define OS_EXIT_CRITICAL() (OSCPURestoreSR(cpu_sr))
```

```
#endif
```

- 第一种，简单的开关中断。
  - 简单高效，但是无法处理嵌套的情况
  - 当有两层临界区保护的时候，在退出内层临界区的时候就会开中断，使外层的临界区也失去保护
- 第二种，引入栈来保存CPSR，判别嵌套中断

- 在OS\_ENTER\_CRITICAL时将中断状态入栈，再关中断
- 但是传参时易出现错误
- 第三种，通过局部变量的方法保存程序当前状态寄存器，再通过局部变量恢复程序当前状态寄存器

## P19 INTERRUPTION

### Event Triggered mechanism

- Classification
  - Interruption 中断
    - *Outside Interruption*
    - *Hard Interruption*
  - Trap 自陷
    - *Inside Interruption(INT 21H、SWI)*
    - *Soft interruption (GDB Debugger)*
  - Exception 异常
    - 除0、访存失败.....
- 中断是一种硬件机制，他是由硬件实现的，因此并不是任何芯片都可以使用中断。只有那些实现了中断机制的芯片才能使用中断。
- 对于软件开发人员来说，主要是在中断的硬件机制上实现中断服务程序。
  - 再进一步，就是就是设计出中断响应服务的抽象层，并通过一定的封装，将中断与通断服务程序结合起来（中断向量号），完成中断服务的注册、执行、返回等。



### • 三种方式处理外界中断请求：

- 中断作为任务切换
  - 触发中断，就触发切换任务，切到一个所关联的任务来处理它
  - 一旦中断发生，与中断关联的任务立即被激活，或者中断释放一个信号量通知同步的任务运行。
  - 这种方式的缺陷是：**中断服务程序需要完成任务切换，而不仅仅是保存被中断任务的上下文，因此中断的开销比较大。**
- 中断作为系统调用
  - 触发中断，中断就立马让RTOS内核去处理它

- 中断作为系统调用也称突发系统调用，就像前后台系统中的突发事务处理一样。系统调用由RTOS内核完成，运行于核心态（用户任务运行于非核心态）中断发生时，如果需要使用系统调用，中断服务程序就向内核发出系统调用请求，由系统调用完成外部中断事务。
- RTOS必须要区分开来系统调用由中断发出还是由任务发出。因为一个任务可以抢占其他任务，但不能抢占中断服务程序。

#### 。 中断作为前台任务

- 最后一种处理中断的方式是将其视为一种特殊的前台任务。这种情况，每个任务都有独立的工作空间，而且内核运行时可以被中断的。
- 中断发生时，**中断服务程序首先保存被中断任务的上下文**，然后判断中断请求的类型。
  - 如果请求能由中断服务程序完成，那么中断服务程序直接进行处理
  - 如果不能，那么就**创建或者激活某个任务来执行**，**中断服务程序将控制权交给内核调度器**，**由它决定触发哪个任务来处理中断请求**，**待该任务完成后，再由中断服务程序恢复中断现场**
- **最终端的处理由中断服务程序和任务共同完成**（操作系统将为中断服务程序提供一些系统调用，以便中断服务程序能够与任务进行通信）

#### 裸板下的ARM9 2440 中断流程

### ARM9 2440的一级中断向量表

University of Electronic Science & Technology of China (UESTC)

|      |                       |
|------|-----------------------|
|      | ⋮                     |
| 0x1C | FIQ                   |
| 0x18 | IRQ                   |
| 0x14 | (Reserved)            |
| 0x10 | Data Abort            |
| 0x0C | Prefetch Abort        |
| 0x08 | Software Interrupt    |
| 0x04 | Undefined Instruction |
| 0x00 | Reset                 |

#### HAL\_VECTR\_START:

```

LDR    pc, VECTOR_TABLE+0    @ Reset
LDR    pc, VECTOR_TABLE+4    @ Undefined
LDR    pc, VECTOR_TABLE+8    @ SWI
LDR    pc, VECTOR_TABLE+0xc   @ Prefetch Abort
LDR    pc, VECTOR_TABLE+0x10  @ Data Abort
LDR    pc, VECTOR_TABLE+0x14  @ RESERVED
LDR    pc, VECTOR_TABLE+0x18 @ IRQ
LDR    pc, VECTOR_TABLE+0x1c  @ FIQ

```

#### VECTOR\_TABLE:

```

.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long IRQ
.long EXP_HANDLER

```

#### HAL\_VECTR\_END:



当开发板复位时，S3C2440 的 ARM920T 内核将当前的 CPSR 与 PC 存入管理模式下的 LR(R14)与 SPSR 中，然后强制将 CPSR 的 M[4:0]位写为 10011（管理模式），并将 CPSR 的 I 位与 F 位置 1(屏蔽 IRQ 与 FIQ)，将 CPSR 的 T 位清 0（进入 ARM 指令模式）。之后，强制将 PC 的值设为 0x0，CPU 从 0x0 开始取值并执行指令。这时 CPU 运行在 ARM 状态。

### （2）未定义指令异常

当 S3C2440A 遇到一个无法处理的指令时，将发生未定义指令异常。未定义异常发生后，下一条指令的地址会被保存到相应的 LR 中，CPSR 被保存到相应的 SPSR 中，CPSR 的 M[4:0]被设置为相应的模式值，并且 PC 被强制赋值为 0x4。这时，CPSR 的中断位可能会被置 1，这里“可能”的意思是，根据不同的异常(除复位异常一定置 1 外)，S3C2440A 芯片会采取不同的行为。如果在这些异常处理中可以中断嵌套，则可以手动将 I 位清零，否则就置 1。以下各个异常均与此类似。

### （3）软中断异常

当 SWI 指令被执行时，将发生软中断异常。当软中断异常发生时，S3C2440A 采取与未定义指令异常类似的措施，只是相应的设置值变为软中断异常所需要的值，PC 取值为 0x8。软中断是由用户模式切换到特权模式的唯一途径。软中断会将程序带入管理模式，这样，程序就对更多的寄存器，特别是 CPSR 有了修改的权利。软中断通常用来实现特权模式下的系统调用功能。

### （4）预取指终止异常

当一条指令的预取指失败(通常为内存读取错误)时，将发生预取指终止异常。S3C2440A 对各寄存器采取与上述异常类似的设置，PC 取值为 0xC。预取指终止异常只有当指令进入了流水线（也就是指令被预取指）时发生，但是如果引发异常的指令没有被执行，则程序不会终止去处理异常，因为之前可能有跳转指令被执行从而跳过了该指令。

### （5）数据终止异常

当在读取数据过程中发生内存错误时，将发生数据终止异常。S3C2440A 采取与上述异常类似的措施，PC 取值为 0x10。当发生数据终止异常时，在异常处理函数中应注意发生变化的寄存器值，详情可参考 S3C2440A 的芯片手册<sup>[44]</sup>。

### （6）IRQ 中断异常

当 CPU 接收到外部设备发出的中断请求时，IRQ 中断异常发生。这时，S3C2440A 采取与上述异常处理类似的措施，PC 取值为 0x18。IRQ 中断异常会在 FIQ 快速中断异常中被屏蔽。在 IRQ 中断异常处理函数中，建议手动指明该中断能否嵌套。

### （7）FIQ 快速中断异常

FIQ 快速中断异常是为数据传输提供的高速中断通道。当 FIQ 快速中断异常发生时，ARM920T 采取与上述类似的措施，PC 取值为 0x1C。FIQ 快速中断异常将进入 FIQ 快速中断模式，在此模式下，ARM 提供了更多的专用寄存器，这样就为中断处理节省了寄存器保护入栈的时间。

## 核心任务

- **目标：保证触发外部中断后，原程序以及中断服务程序能正常运转**
  - **书写IRQ的中断服务程序**
    - 跳转到哪儿去？
    - 返回到哪儿去？

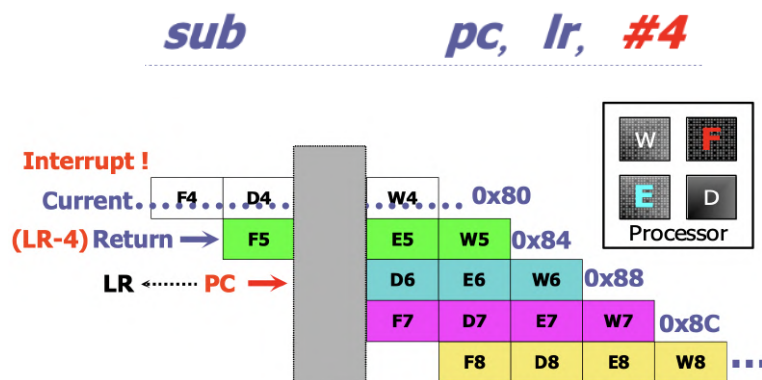
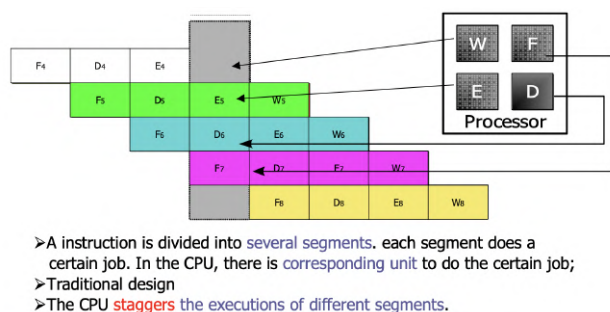
- 上下文怎么保存？
- 上μc/OS II后？
- .....

## Q1 Where to return

触发中断时，硬件自动将PC置为对应的中断向量表。但执行完中断服务程序后，返回的地址在哪里，怎么返回？

- 跳转到中断/**执行**跳转时，LR自动得到PC的备份。
- 但为什么返回的时候LR要减去4才能返回给PC？
  - 我们希望返回到 *Current* 的下一条（Current+4）
  - 但是，在执行Current时，PC已经变成Current+8啦！所以LR也是Current+8
  - 这是因为——

### • CPU的流水线作业



错开各个指令的执行，让W、F、E、D随时随刻都在工作

当前指令还没执行完时，PC已经在往后取新指令了

当一个中断（异常）发生时，处理器会自动在 LR（R14）中保存一个与当前 PC 值相关的信息，中断服务程序可以利用这个信息来推算返回地址。具体如表 2-10 所示。

表 2-10 中断（异常）返回地址

| 异常/中断          | 地址   | 说明                |
|----------------|------|-------------------|
| 复位中断（RESET）    | —    | —                 |
| 未定义指令异常（UNDEF） | LR   | 指向未定义指令的下一条指令     |
| 软中断异常（SWI）     | LR   | 指向 SWI 指令的下一条指令   |
| 预取指令中止异常（PABT） | LR-4 | 指向导致预取指令中止异常的那条指令 |
| 数据中止异常（DABT）   | LR-8 | 指向导致数据中止异常的那条指令   |
| 普通中断（IRQ）      | LR-4 | IRQ 处理程序的返回地址     |
| 快速中断（FIQ）      | LR-4 | FIQ 处理程序的返回地址     |

中断（异常）处理完毕之后，ARM 处理器会执行以下几步操作从中断（异常）中返回。

- (1) 将连接寄存器（LR）的值减去相应的偏移量后送到 PC 中。
  - (2) 将 SPSR 赋值给 CPSR。
  - (3) 若在进入中断（异常）处理时设置了中断禁止位，则要在此清除。
- 一个非嵌套中断的响应与处理流程图如图 2-25 所示。其框架代码如下。

```
IRQ_handler
SUB  R14,R14,#4
STMFD R13!,{R0-R3,R12,R14}
...
LDMFD R13!,{R0-R3,R12,PC}^
```

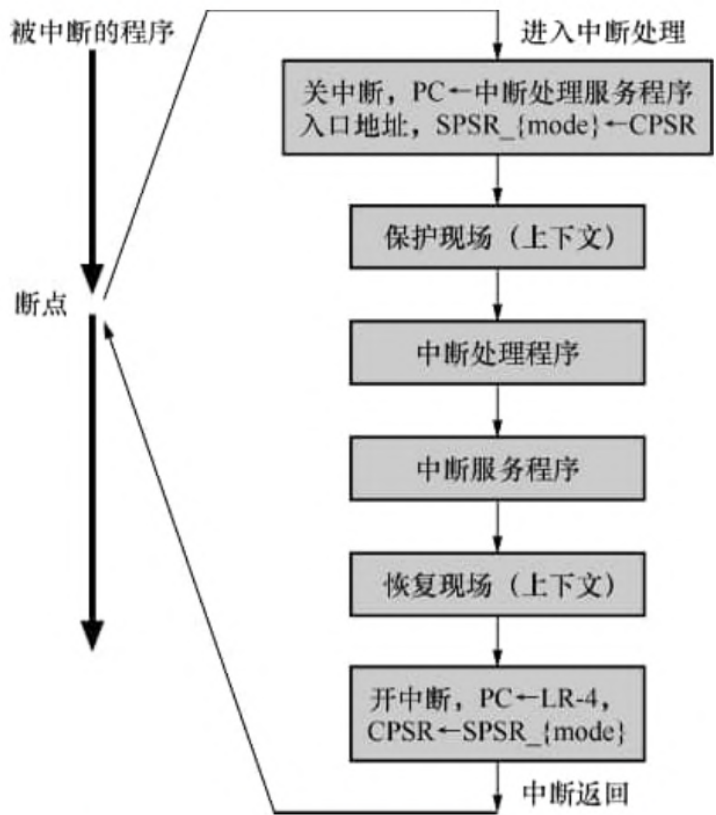


图 2-25 非嵌套中断的响应与处理流程图

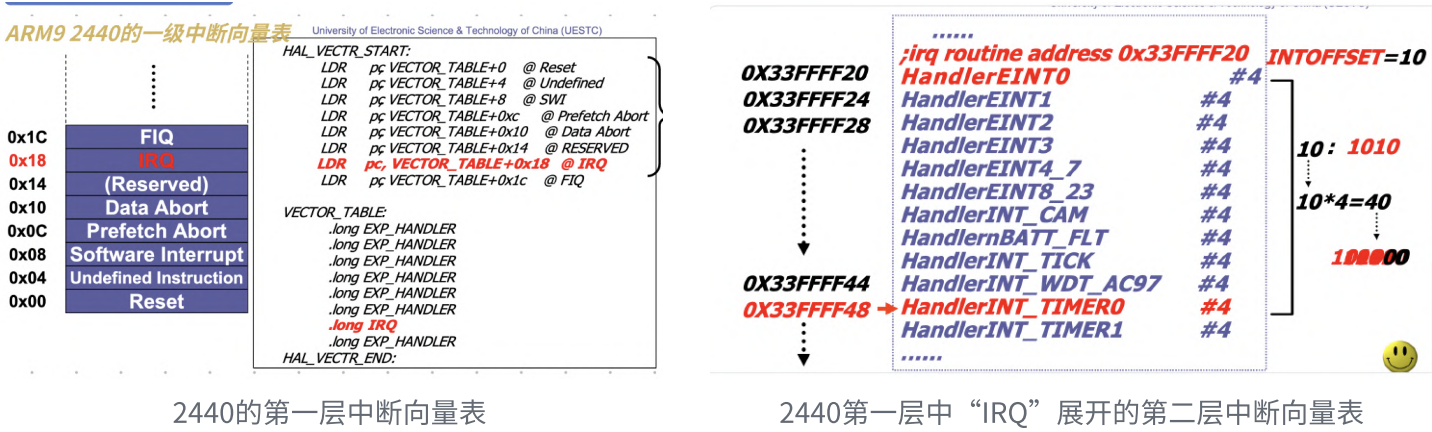
• Solution：在最后一行加上

```
代码块
1 SUB PC, LR, #4
```

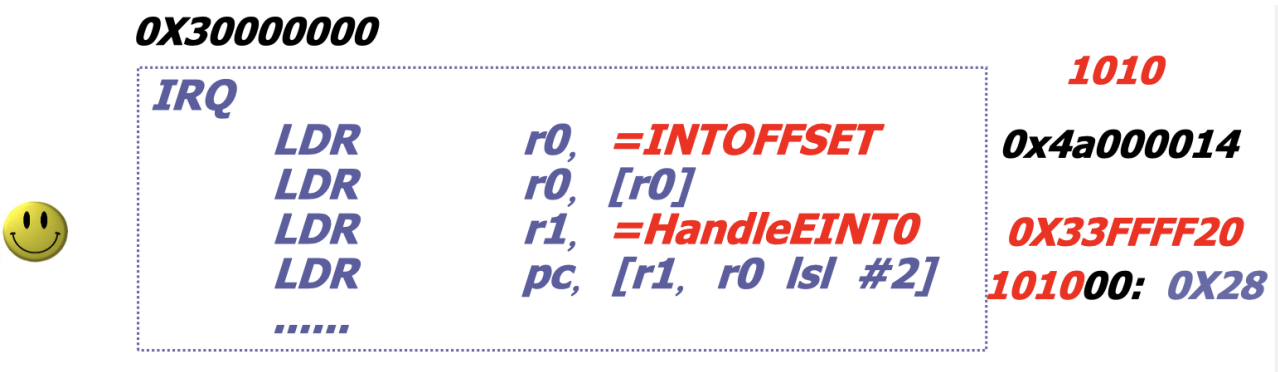
Q2 How to distinguish different INTs?

STM32中只有一个层级的中断，而2440中有两层中断——第一层就是上面的RESET....，第二层却存储在RAM中。

- 左图是第一层中断，也称为公共入口；对于IRQ，经过第一层中断后就要被导向到第二层中断来 **distinguish different INTs**。
- 第一层中断向量表是所有中断和异常的入口，他由数条LDR指令组成。不同的终端、异常触发时会执行到不同的LDR指令。
- 触发IRQ中断时，便会进入第一层中断向量表，随后跳转到IRQ中断服务程序。在IRQ服务程序中，再跳转到第二层中断向量表



- 在三星2440中，有一个类似于STM32中NVIC的终端管理器，它能集中管理外部中断源，并通过INTOFFSET寄存器来告诉CPU外边出发了哪个中断。于是，我们在IRQ中断服务程序中书写下面的代码来 **distinguish different INTs**
- Solution:



- 硬件自动根据中断源，确定该执行哪个中断（32中是由NVIC完成）

### Q3 How to save the information of interrupted programs?

- 压栈R0~R12
- 压栈R14（LR）
- 压栈CPSR
  - 通过SPSR&R0



## SVC模式



## Q5 [ OS ] How to process context switch triggered by the IRQ?

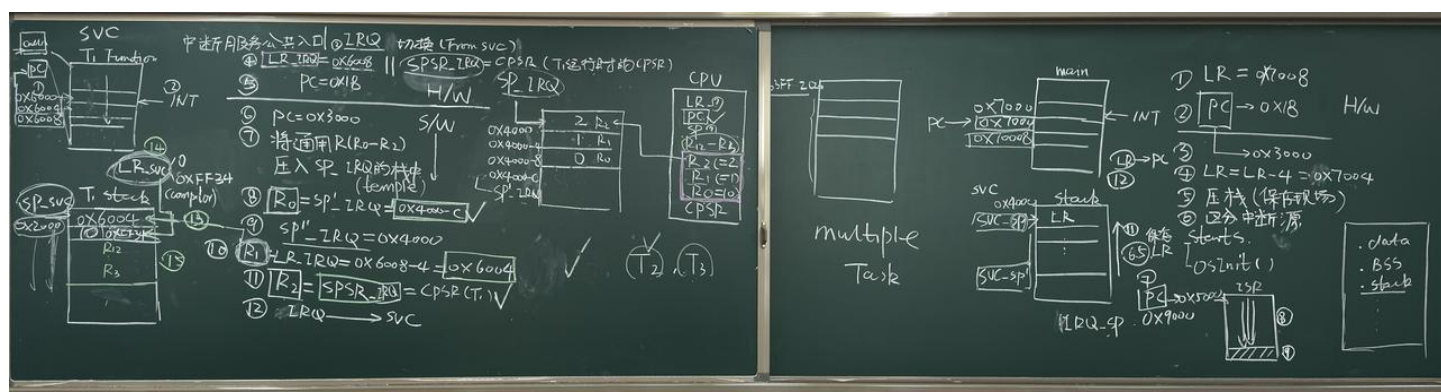
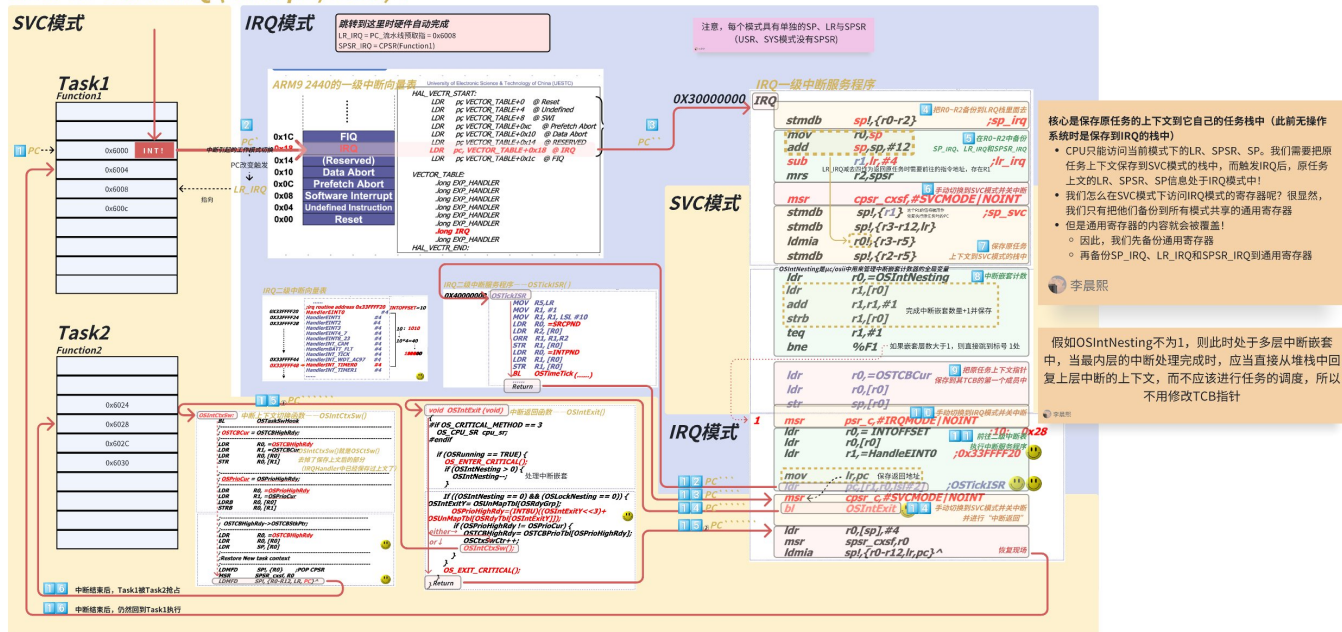
## ——中断结束后的返回点不同了！

- 没有操作系统时，所有代码均为一个线程。不管怎么弄跳转都在一个线程里面跳
- 有操作系统时，跳转就牵扯到任务（线程）的切换了！被中断的线程不一定会在中断结束后立即恢复执行。这时候要保证被中断的线程能再度从被中断处继续执行，因此多了很多操作。

- $\mu\text{C}/\text{OS II}$ 是一个可抢占的内核，**中断处理完成之后，内核调度程序OS\_Sched()会从就绪队列中找到优先级最高的任务运行（优先级最高的任务并不一定是被中断的那个任务）**，因此在发生中断、进入用户的中断服务程序前，首先需要保存被中断任务的上下文。
- 此外， $\mu\text{C}/\text{OS II}$ 允许中断嵌套，所以在中断服务程序中还需要增加中断嵌套计数器，然后根据嵌套数量来决定是否重新调度。

这些就是μC/OS II中的中断与裸板中断的区别。

- 所有的IRQ中断都会进入同一个公共入口，于是乎我们可以效仿上面的做法。把IRQ中断的公共操作都放在这里面，比如：
  - 保存原任务的上下文到它的栈中
  - 增加中断嵌套计数器
  - 确定对应的中断向量号
  - 完成中断服务后，调用 `uc/OS II` 的调度函数



#### 4.8 OS（左）与裸板（右）的中断流程

### Q6 Is it an address in each vector?

很显然，它不是地址，而是LDR指令。

|      |                       |                                                                                                                                                                               |
|------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | ⋮                     |                                                                                                                                                                               |
| 0x1C | FIQ                   | HAL_VECTR_START:<br>LDR pc VECTOR_TABLE+0 @ Reset                                                                                                                             |
| 0x18 | IRQ                   | LDR pc VECTOR_TABLE+4 @ Undefined                                                                                                                                             |
| 0x14 | (Reserved)            | LDR pc VECTOR_TABLE+8 @ SWI                                                                                                                                                   |
| 0x10 | Data Abort            | LDR pc VECTOR_TABLE+0xc @ Prefetch Abort                                                                                                                                      |
| 0x0C | Prefetch Abort        | LDR pc VECTOR_TABLE+0x10 @ Data Abort                                                                                                                                         |
| 0x08 | Software Interrupt    | LDR pc VECTOR_TABLE+0x14 @ RESERVED                                                                                                                                           |
| 0x04 | Undefined Instruction | LDR pc, VECTOR_TABLE+0x18 @ IRQ                                                                                                                                               |
| 0x00 | Reset                 | LDR pc VECTOR_TABLE+0x1c @ FIQ                                                                                                                                                |
|      |                       | VECTOR_TABLE:<br>.long EXP_HANDLER<br>.long EXP_HANDLER<br>.long EXP_HANDLER<br>.long EXP_HANDLER<br>.long EXP_HANDLER<br>.long EXP_HANDLER<br>.long IRQ<br>.long EXP_HANDLER |
|      |                       | HAL_VECTR_END:                                                                                                                                                                |

2440的第一层中断向量表

## P21 TIME MANAGEMENT

Time management is important!

- 通过时钟可以实现：
  - 延时任务
  - 周期性触发任务执行
  - 任务有限等待的计时
  - 软定时器的定时管理
  - 确认超时
  - 时间相关的调度操作

### ✓ 大多数嵌入式系统有两种时间源：

#### 1. 实时时钟 (RTC, Real-Time Clock)

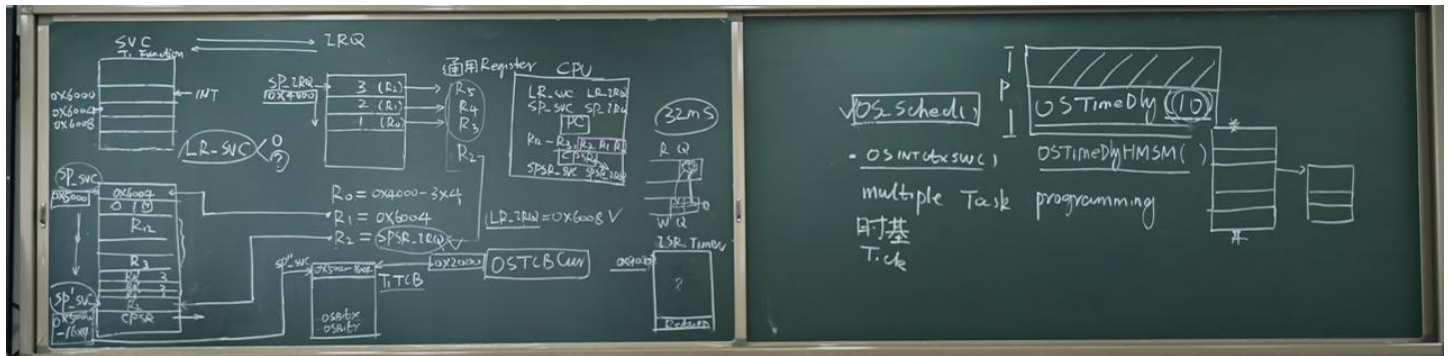
- 依靠电池供电，即使系统断电，也可以维持新日期和时间
- 独立于操作系统，因此也称为硬件时钟，它为整个系统提供一个时间标准

#### 2. 定时器/计数器 System Timer

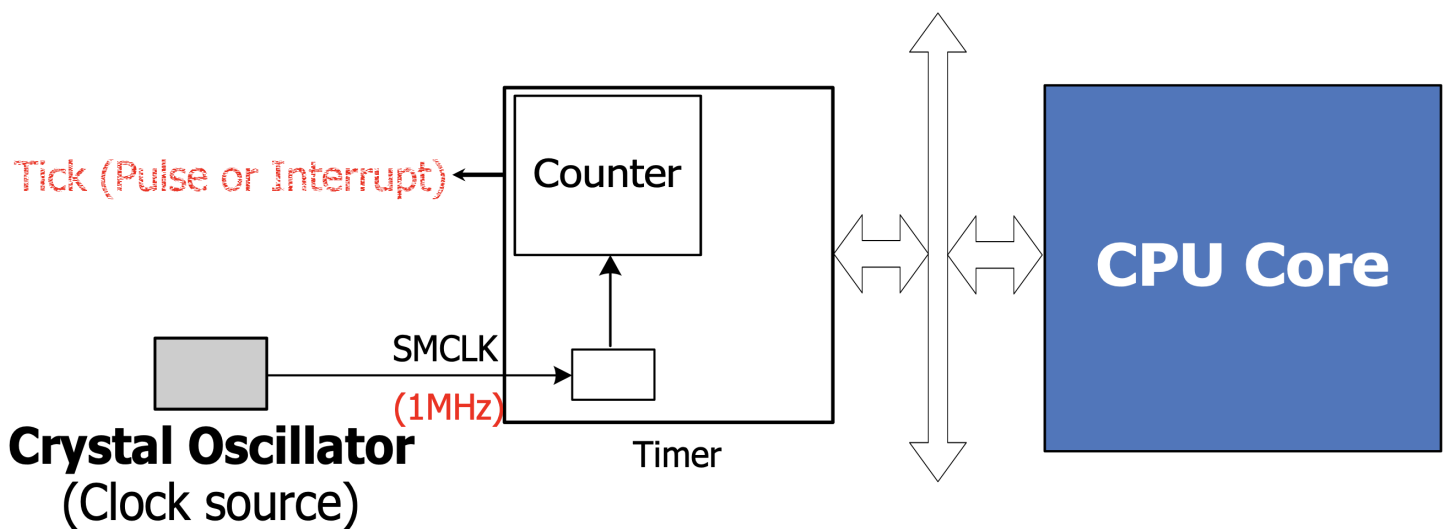
- 嵌入式处理器集成了多个定时器或者计数器
- 实时内核需要一个定时器作为系统时钟，并由内核控制系统时钟工作

- 系统时钟的最小粒度是由应用和操作系统的特点决定的
- Size of System Timer: Tick**

一般而言，实时时钟是系统时钟的基准，实时内核通过读取实时时钟来初始化系统时钟，此后两者保持同步运行，共同维持系统时间。



4.10 有 $\mu\text{c}/\text{OS II}$ 的IRQ服务程序（左）OSTimeDly()函数（右）



System Timer示意图

Q1 Task Delay(  $\mu\text{c}/\text{OS II}$ : **OSTimeDly()** )



## OSTimeDly()

```
void OSTimeDly (INT16U ticks)
```

```
{
```

```
    #if OS_CRITICAL_METHOD == 3
```

```
        OS_CPU_SR cpu_sr;
```

```
    #endif
```

1 临界区保护

```
    if (ticks > 0) Tick大于0才会延时!
```



```
    { OS_ENTER_CRITICAL();
```

```
        if ((OSRdyTbl[OSTCBCur->OSTCBY] &= ~OSTCBCur->OSTCBBitX) == 0)
```

```
            { OSRdyGrp &= ~OSTCBCur->OSTCBBitY; }
```

```
            OSTCBCur->OSTCBDly = ticks;
```

2 将当前任务移除就绪队列

```
    OS_EXIT_CRITICAL();
```

```
    OS_Sched();
```

3 重调度

```
}
```

```
}
```



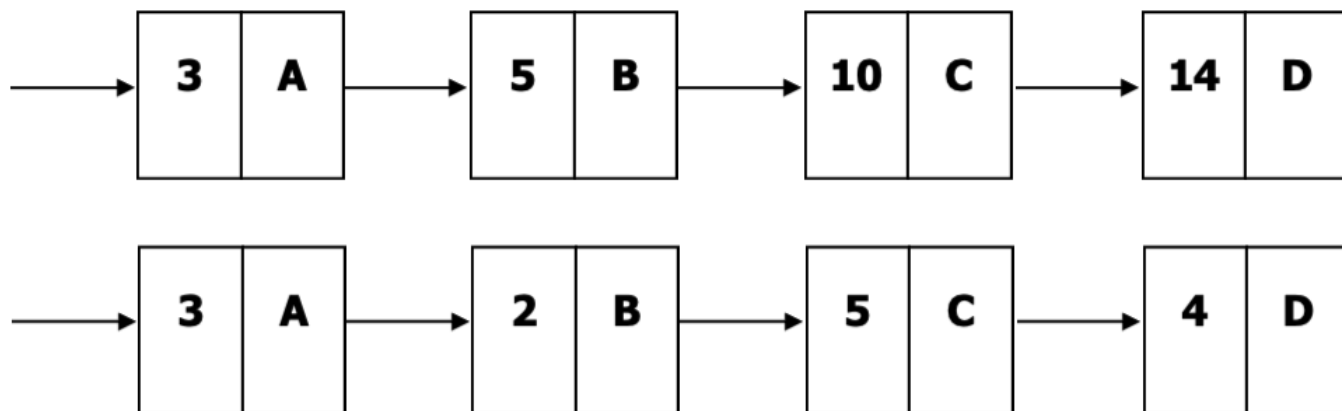
## Q2 Difference Time Chain

为了确保RTOS各项功能执行时间的确定性，需要对内核的各项数据结构进行优化。比如采用差分时间链、双向链表、优先级位图法等。

实时系统在运行过程中，需要进行各种与时间相关的操作。例如， $\mu\text{C}/\text{OS II}$ 通过OSTimeDly()或OSTimeDlyHMSM()实现对任务的延迟操作。若对多个任务进行延迟操作，就构成了一个延迟队列，内核将延迟任务的TCB依次挂载到延迟队列中。例如，用OSTimeDly()将任务A~任务D分别延迟3、5、10和14个ticks:



每当时钟中断产生一个tick，则需要对时间等待链中的每一个结点进行“减1”操作：若时间等待链中的结点数量较多，则时钟中断服务程序的计算开销就比较大。



为了减小计算开销，可采用差分时间等待链来描述延迟队列，如图 3.33 所示，队列中某个结点的值是相对于前一个结点的时间差。例如，任务 B 需要延迟 5 个 ticks，其前一个结点任务 A 需要延迟 3 个 ticks，意味着任务 B 在任务 A 延迟结束后，还需要延迟 2 个 ticks，因此任务 B 所在结点的值为 2，该值是相对于任务 A 的时间差。采用差分时间等待链后，每当时钟中断产生一个 tick，只需对队列头部结点进行“减 1”操作，当减到 0 时，就将其从等待链中取出，后续结点将成为新的头部结点并且被激活。在该过程中，等待链中其他结点的值保持不变，无须对每一个结点进行“减 1”操作，这样可减小计算开销。

- *How to insert an event into the chain?*

- 如有新任务要进行延迟操作，需要在差分时间等待链中插入新的结点。例如，任务 E 要延迟 7 个 ticks，如图 3.34 所示，这样只需在任务 B 和任务 C 之间插入任务 E ( $3+2 < 7 < 3+2+5$ )



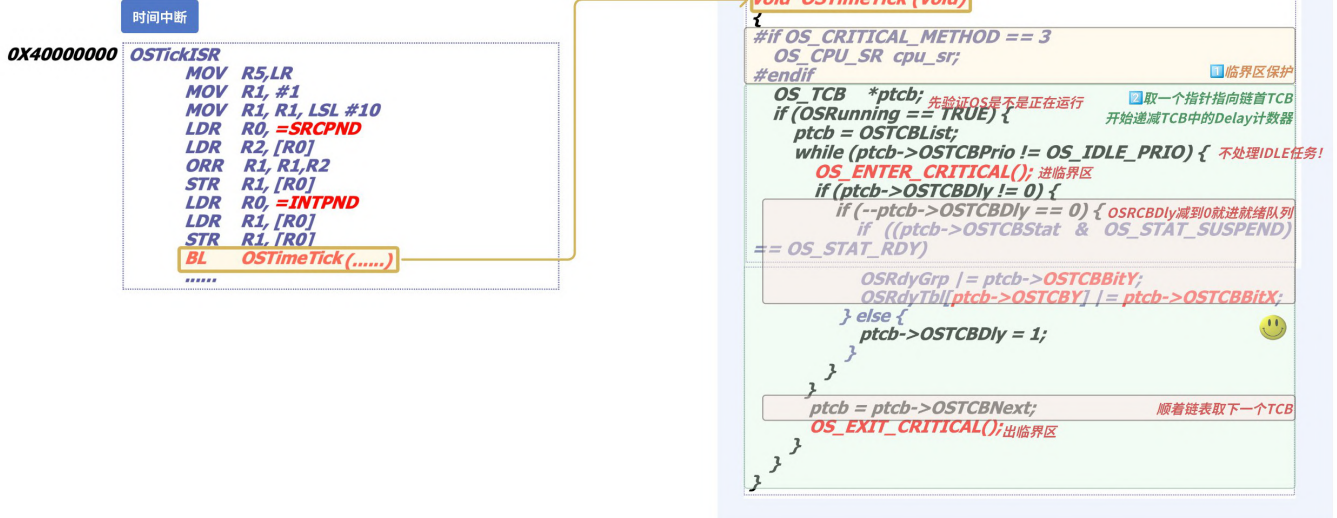
### Q3 WatchDog Timer

```
wdId=wdCreate()
```

```
wdStar(wdId,sysClkRateGet(), DEADLINE_TIME, (FUNCTION)deadlineHandler)
```

### Watchdog timer

### Q4 System Clock ISR



## - 3 Hardware System -

1

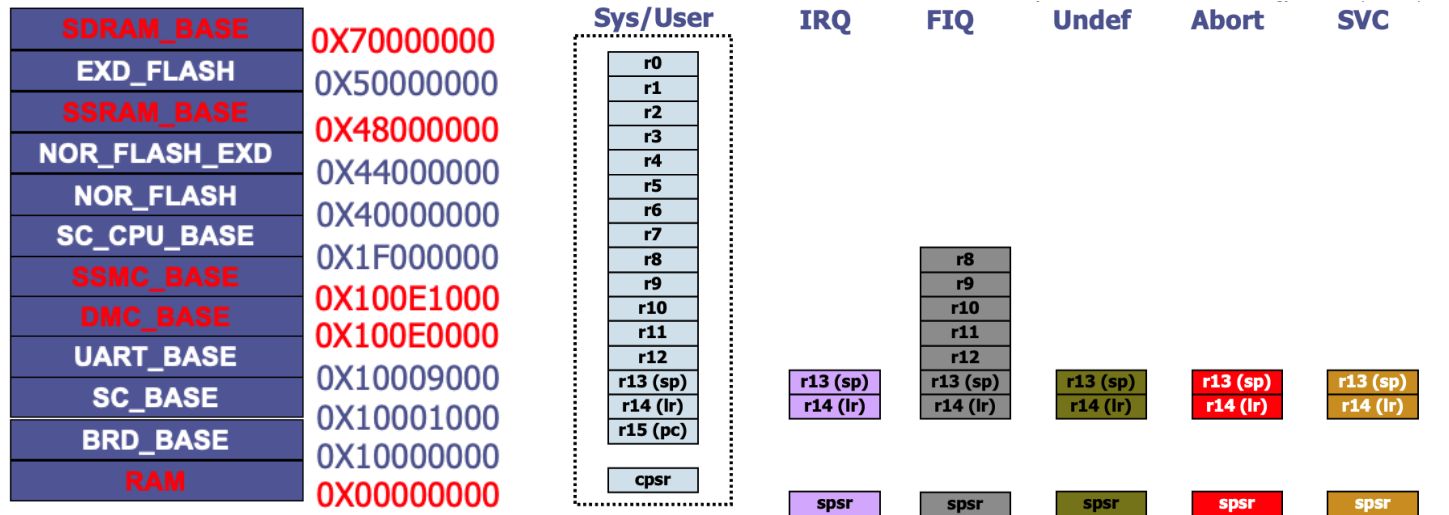
University of Electronic Science & Technology of China (UESTC)

# Embedded Operating System

LIAO Yong

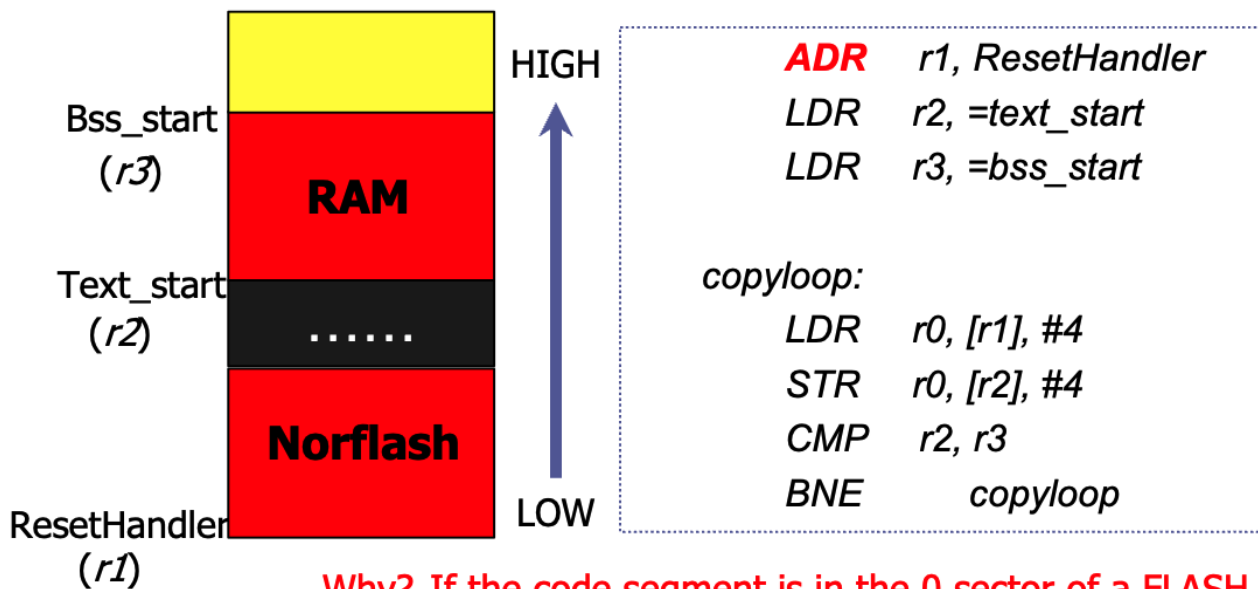
“我给你们讲的都是没用的东西，詹老师讲的才是有用的东西。”

## Memory Layout



## Copy Code

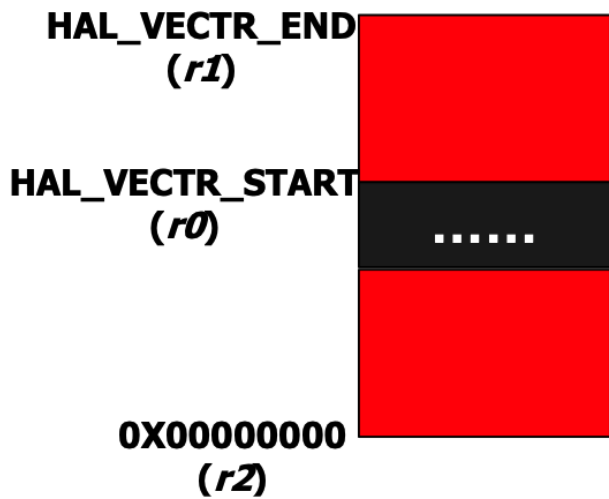
### Practice1: How to copy a code segment to RAM ?



## Copy Vectors



## Practice2: How to copy interrupt vectors to address 0 ?



**copy\_vectors:**

```

LDR  r0, =HAL_VECTR_START
CMP  r0, #0
BEQ  real_code
LDR  r1, =HAL_VECTR_END
MOV  r2, #0

```

**1:**

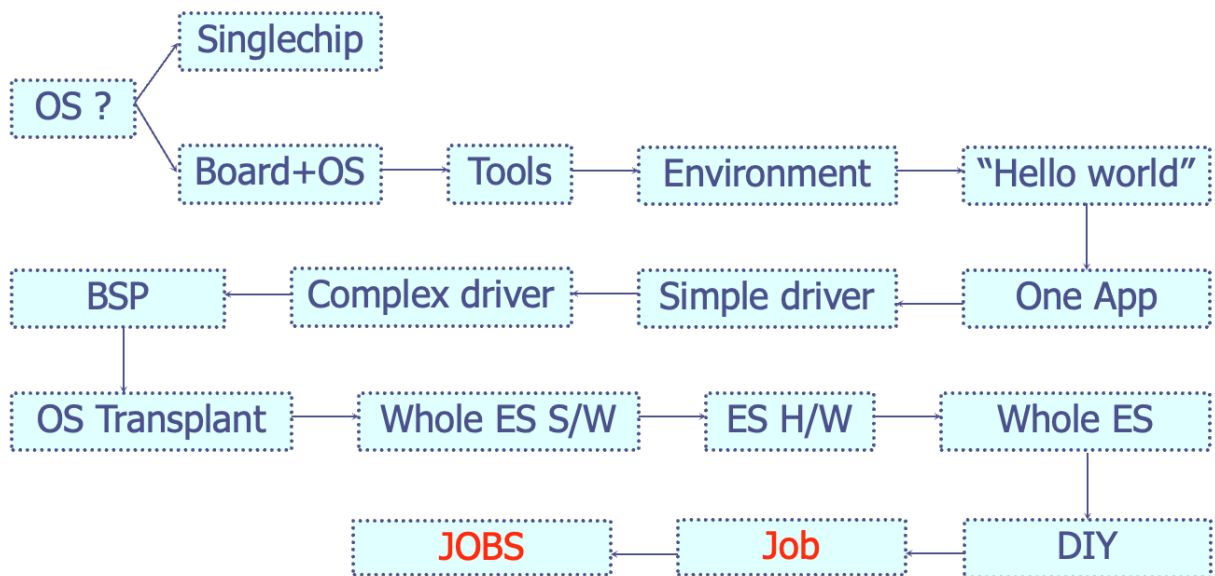
```

LDR  r3, [r0], #4
STR  r3, [r2], #4
CMP  r0, r1
BL   1b

```



How to find good jobs?



- 4 Software System -

# Embedded Operating System

LIAO Yong

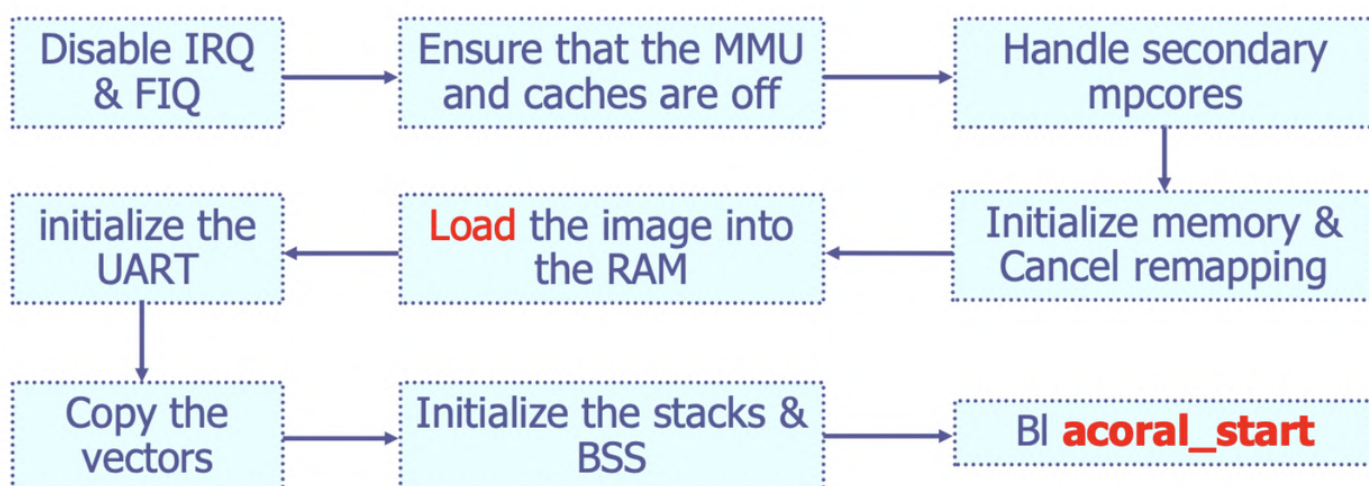
Real-Time Systems Lab

LIAO Yong

P 4 Software System.ppt

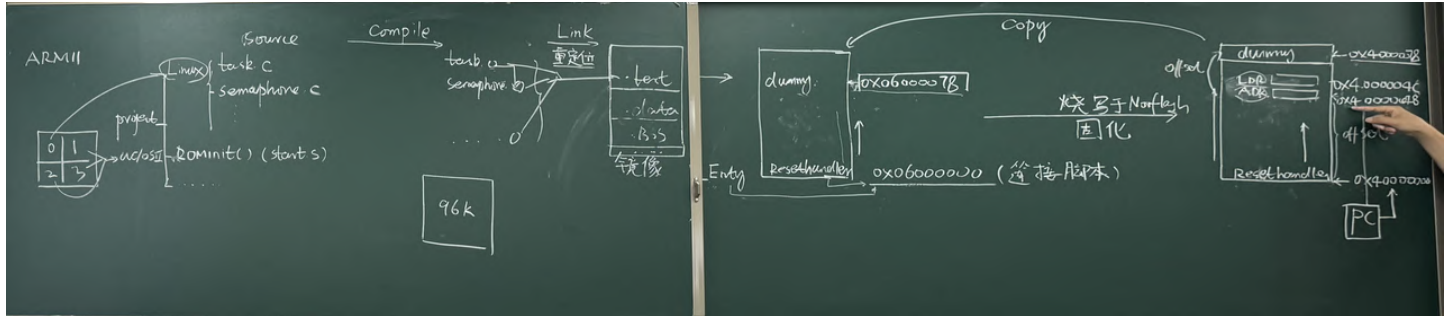
## ARM 启动流程

代码参加附录的BootLoader.s

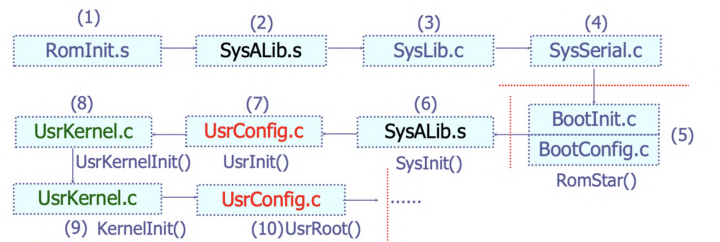
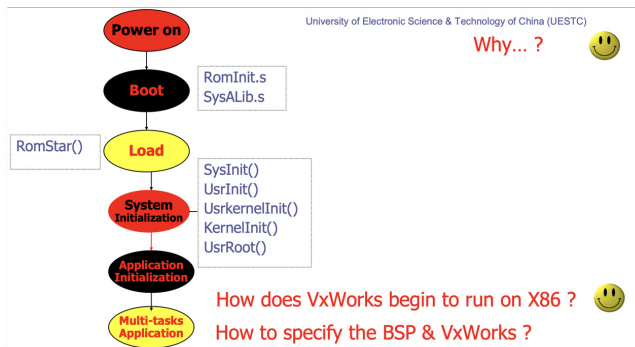


- 重点代码
  - Load the image into the RAM

- Copy Vectors
- Initialize the BSS data



## X86 启动流程



How to compile & link files in the project in the **Linux**? 😊😊

## RomInit.s

```
%include "pm.inc"
org F0000h
jmp LABEL_BEGIN
```

[SECTION .gdt]

|                    | Base                | Limit             | Attribution  |
|--------------------|---------------------|-------------------|--------------|
| LABEL_GDT:         | Descriptor 0,       | 0,                | 0            |
| LABEL_DESC_CODE32: | Descriptor 0,       | SegCode32Len - 1, | DA_C + DA_32 |
| LABEL_DESC_VIDEO:  | Descriptor 0B8000h, | 0ffffh,           | DA_DRW       |

```
GdtLen equ $ - LABEL_GDT
GdtPtr dw GdtLen - 1
dd 0
```

```
SelectorCode32 equ LABEL_DESC_CODE32 - LABEL_GDT
SelectorVideo equ LABEL_DESC_VIDEO - LABEL_GDT
; END of [SECTION .gdt]
```

先是有个Description Tabel

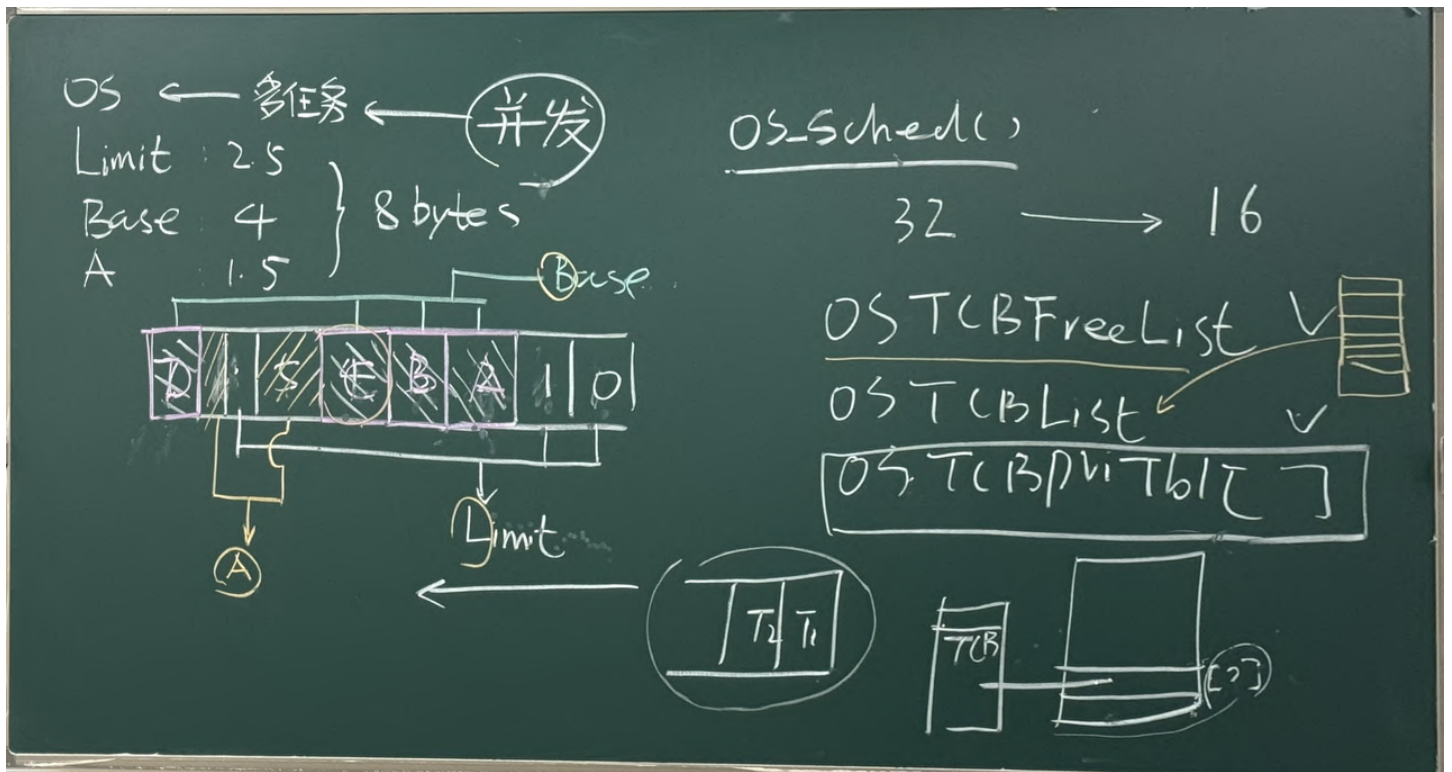
然后执行一段16位格式的代码

随后进行“实模式-》保护模式”的切换，开始执行32位格式的代码









4.22 X86启动流程

## 附录

### S3C2440 BootLoader

```

#include "hal_brd_cfg.h"
.global __ENTRY
.global ResetHandler
__ENTRY:
ResetHandler:
    MRS        r0, CPSR
    ORR        r0, r0, #(PSR_I_BIT|PSR_F_BIT)    @ Disable IRQ & FIQ
    MSR        CPSR_c, r0

    @
    @ Ensure that the MMU and caches are off
    @
    MOV        r0, #0
    MCR        p15, 0, r0, c7, c5, 0            @ Invalidate I Cache
    MCR        p15, 0, r0, c7, c6, 0            @ Invalidate D Cache

    MRC        p15, 0, r0, c1, c0, 0            @ Get control register
    BIC        r0, r0, #(CTRL_M_BIT|CTRL_C_BIT) @ Disable MMU and D Cache
    BIC        r0, r0, #CTRL_I_BIT              @ Disable I Cache
    MCR        p15, 0, r0, c1, c0, 0            @ Write control register

    @
    @ Handle secondary mpcores
    @
    MRC        p15, 0, r0, c0, c0, 5
    ANDS       r0, r0, #0x0f
    BEQ        clear_leds                       @ Go if core 0 on primary core tile
    BL         __secondary_mpcore              @ Will not return

    @
    @ Clear the LED s
    @
clear_leds:
    MOV        r0, #BRD_BASE
    MOV        r1, #0
    STR        r1, [r0, #BRD_LED]

    @
    @ Initialise memory
    @
    BL         __platform_memory_init

    @
    @ Are we running from our load address
    @
    LDR        r4, =dummy
    ADR        r1, dummy
    CMP        r1, r4
    BXEQ       r4

    @
    @ Get ready to copy ourselves to our execution address

```

## 期中随堂测验 2024 (90 分钟)

- 1、 $\mu\text{C}/\text{OS II}$  创建任务过程中定义的“\* OSTCBLIST”和“\* OSTCBFREELIST”分别有什么作用？他们与任务就绪队列有什么关系？任务 TCB 中“OSTCBBITY”、“OSTCBBITX”、“OSTCBY”、“OSTCBX”在任务调度中起到什么作用，他们的值如何确定？（17 分）
- 2、任务上下文切换的主要内容是什么？请提供  $\mu\text{C}/\text{OS-II}$  在 ARM mini2440 上任务切换的代码并进行解释。任务上下文切换函数 OS\_TASK\_SW() 中，为什么会两次保存 LR？这两个 LR 有什么不同？（20 分）
- 3、 $\mu\text{C}/\text{OS-II}$  如何利用优先级位图法对调度过程进行实时化和确定化？请给出实现代码，并详细解释其实现原理。（23 分）
- 4、主流的嵌入式操作系统有哪些？对其中的 2 种分别进行进一步解释，给出其行业应用，说明他们是如何根据行业应用需求进行专门设计或者优化。（10 分）
- 5、请叙述 ARM9 2440 裸板中断（IRQ）处理流程。中断响应和处理器时，需要首先将什么信息进行保存？为什么？该过程是由硬件实现还是软件实现？外部中断触发后，CPU 是如何去区分并执行相应的中断服务程序的？（17 分）
- 6、当你在嵌入式平台上开发计算密集型应用时，如果遇到了性能瓶颈，可从哪些角度出发提升代码的运行速度？请进行简要说明（13）

## 24年期末试题回忆

### 1. 差分时间链

- 根据题目描述画一条差分时间链出来
- 插入一个新任务后，画出新的差分时间链
- 差分时间链的设计有什么优点

### 2. 任务有什么特征，任务与进程的区别，任务与函数的区别

- 概念题

### 3. CPSR寄存器的N、Z、C、V分别的作用是什么，请根据图示汇编代码说说是如何依次提高代码运行效率的。

### 4. ERTOS采用了什么措施来保证它的实时性与确定性？

### 5. 给出 $\mu\text{C}/\text{OS II}$ 中IRQ的中断处理代码，让你根据代码讲解有操作系统的时候的IRQ处理流程

### 6. 这道题忘了呜呜

### 7. 给了一个点灯一个灭灯的Task（就是上课PPT的那两个Task），让你画OSTimeDly(10)函数的流程图

### 8. 优先级位图法

- 写了代码

- 解释你写的代码
- 给你一些插入任务的情景，让你写出每插入一个任务后优先级位图的变化